



*Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

**Tema 64. El modelo relacional. El lenguaje SQL. Normas y
estándares para la interoperabilidad entre gestores de bases
de datos relacionales**

AUTOR: José Campos

Actualización 2015

ÍNDICE

1	EL MODELO RELACIONAL.....	5
1.1	INTRODUCCIÓN.....	5
1.2	CARACTERÍSTICAS Y CONCEPTOS BÁSICOS	5
1.2.1	DEFINICIONES	5
1.2.2	VALORES NULOS	6
1.2.3	RESTRICCIONES	7
1.2.3.1	RESTRICCIÓN DE DOMINIO	7
1.2.3.2	RESTRICCIÓN DE CLAVE	8
1.2.3.3	RESTRICCIÓN DE IDENTIDAD DE LA ENTIDAD	9
1.2.3.4	RESTRICCIÓN DE INTEGRIDAD REFERENCIAL	9
1.2.4	COMPONENTES DE UNA BASE DE DATOS RELACIONAL	10
1.3	REGLAS DE CODD	10
1.3.1	REGLA 0 (REGLA FUNDAMENTAL).....	10
1.3.2	REGLA 1 (DE LA INFORMACIÓN).....	10
1.3.3	REGLA 2 (DEL ACCESO GARANTIZADO).....	10
1.3.4	REGLA 3 (DEL TRATAMIENTO SISTEMÁTICO DE LOS VALORES NULOS)	10
1.3.5	REGLA 4 (CATÁLOGO EN LÍNEA DINÁMICO BASADO EN EL MODELO RELACIONAL)....	10
1.3.6	REGLA 5 (DEL SUBLENGUAJE COMPLETO DE DATOS)	11
1.3.7	REGLA 6 (DE ACTUALIZACIÓN DE VISTAS)	11
1.3.8	REGLA 7 (INSERCIÓN, ACTUALIZACIÓN Y BORRADO DE ALTO NIVEL)	11
1.3.9	REGLA 8 (INDEPENDENCIA FÍSICA DE DATOS)	11
1.3.10	REGLA 9 (INDEPENDENCIA LÓGICA DE DATOS)	11
1.3.11	REGLA 10 (INDEPENDENCIA DE INTEGRIDAD)	11
1.3.12	REGLA 11 (INDEPENDENCIA DE DISTRIBUCIÓN)	11
1.3.13	REGLA 12 (DE LA NO SUBVERSIÓN)	12
1.4	NORMALIZACIÓN	12
1.4.1	CONCEPTOS PREVIOS	12
1.4.1.1	DEPENDENCIA FUNCIONAL.....	12
1.4.1.2	DEPENDENCIA FUNCIONAL COMPLETA	12
1.4.1.3	DEPENDENCIA TRANSITIVA	13
1.4.1.4	DEPENDENCIA MULTIVALUADA	13
1.4.2	PRIMERA FORMA NORMAL (1FN).....	14
1.4.3	SEGUNDA FORMA NORMAL (2FN).....	14
1.4.4	TERCERA FORMA NORMAL (3FN)	15
1.4.5	FORMA NORMAL DE BOYCE-CODD (FNBC)	15
1.4.6	CUARTA FORMA NORMAL (4FN).....	16
1.4.7	QUINTA FORMA NORMAL (5FN).....	17
1.4.8	SEXTA FORMA NORMAL (6FN)	18
1.5	BASE MATEMÁTICA DEL MODELO RELACIONAL. LENGUAJES RELACIONALES ...	20
1.5.1	ÁLGEBRA RELACIONAL.....	20
1.5.1.1	SELECCIÓN (Σ).....	20
1.5.1.2	PROYECCIÓN (Π)	21
1.5.1.3	UNIÓN ($\cup$).....	21
1.5.1.4	DIFERENCIA DE CONJUNTOS (-).....	21
1.5.1.5	PRODUCTO CARTESIANO ($\times$).....	21
1.5.1.6	RENOMBRAMIENTO (P)	22
1.5.1.7	OTRAS OPERACIONES ADICIONALES	22
1.5.2	CÁLCULO RELACIONAL DE TUPLAS.....	25
1.5.2.1	SELECCIÓN	25
1.5.2.2	PROYECCIÓN	26
1.5.2.3	UNIÓN.....	26

1.5.2.4	DIFERENCIA DE CONJUNTOS	26
1.5.2.5	PRODUCTO CARTESIANO	26
1.6	CONCLUSIONES	26
2	EL LENGUAJE SQL	27
2.1	INTRODUCCIÓN Y BREVE REFERENCIA HISTÓRICA	27
2.2	ESTANDARIZACIÓN Y VERSIONES	27
2.3	ELEMENTOS DEL LENGUAJE	27
2.3.1	TIPOS DE DATOS	28
2.3.2	OPERADORES Y EXPRESIONES CONDICIONALES	28
2.3.3	CONSULTAS (QUERIES)	29
2.3.4	LENGUAJE DE MANIPULACIÓN DE DATOS (DML).....	31
2.3.4.1	INSERCIÓN DE DATOS (INSERT).....	31
2.3.4.2	ACTUALIZACIÓN DE DATOS (UPDATE).....	32
2.3.4.3	ACTUALIZACIÓN E INSERCIÓN SIMULTÁNEAS REFERIDAS A OTRA TABLA (MERGE)	32
2.3.4.4	BORRADO DE DATOS (DELETE Y TRUNCATE).....	33
2.3.4.5	CONFIRMACIÓN Y MARCHA ATRÁS EN SENTENCIAS DML.....	33
2.3.5	LENGUAJE DE DEFINICIÓN DE DATOS (DDL).....	33
2.3.5.1	CREACIÓN, MODIFICACIÓN Y ELIMINACIÓN DE TABLAS	33
2.3.5.2	CREACIÓN, MODIFICACIÓN Y ELIMINACIÓN DE VISTAS.....	34
2.3.5.3	CREACIÓN Y ELIMINACIÓN DE ÍNDICES	35
2.3.5.4	CREACIÓN Y ELIMINACIÓN DE REGLAS DE INTEGRIDAD	35
2.3.6	LENGUAJE DE CONTROL DE DATOS (DCL)	35
2.4	EXTENSIONES PROCEDURALES.....	36
2.5	CONCLUSIONES	36
3	NORMAS Y ESTÁNDARES PARA LA INTEROPERABILIDAD ENTRE GESTORES DE BASES DE DATOS RELACIONALES	37
3.1	INTRODUCCIÓN.....	37
3.2	PRIMEROS ESFUERZOS Y APARICIÓN DE ODBC	37
3.3	OLE DB (OBJECT LINKING AND EMBEDDING DATABASE), ADO (ACTIVE X DATA OBJECTS) Y ADO.NET	37
3.4	JDBC (JAVA DATABASE CONNECTIVITY)	38
3.4.1	TIPO 1: PUENTE JDBC-ODBC	38
3.4.2	TIPO 2: API NATIVA	38
3.4.3	TIPO 3: PROTOCOLO DE RED.....	39
3.4.4	TIPO 4: JAVA PURO	40
3.5	CONCLUSIONES	40
4	PRINCIPALES GESTORES DE BASES DE DATOS RELACIONALES	41
4.1	FABRICANTE, ÚLTIMA VERSIÓN Y TIPO DE LICENCIA	41
4.2	SISTEMAS OPERATIVOS PRINCIPALES EN LOS QUE ESTÁ DISPONIBLE	41
4.3	LÍMITES DE TAMAÑO DE LA PROPIA BASE DE DATOS	41
4.4	LÍMITES DE TAMAÑO DE LOS TIPOS DE DATOS PRINCIPALES	41
4.5	CONCLUSIONES	42

GLOSARIO.....	43
PREGUNTAS DE TEST.....	44
SOLUCIONES A LAS PREGUNTAS DE TEST.....	47
BIBLIOGRAFÍA	48

1 El Modelo Relacional

1.1 Introducción

El modelo relacional fue propuesto originalmente por Edgar F. Codd en 1970. El principal objetivo de su propuesta era sentar las bases para el diseño y la implementación de Sistemas de Bases de Datos en los que las aplicaciones que accedieran o manipularan la información almacenada fueran totalmente independientes de cómo se implementara dicho almacenamiento, es decir, conseguir dentro del mundo de las bases de datos la separación del punto de vista físico (almacenamiento de la información) y el punto de vista lógico (tratamiento de la misma)

En los modelos tradicionales existentes en el momento en que Codd realiza su propuesta, fundamentalmente el modelo en red y el modelo jerárquico, el diseño físico y el modelo lógico estaban íntimamente relacionados, lo que tiene consecuencias no deseables que la propuesta de Codd permitía superar, entre otras:

- Necesidad de un conocimiento detallado del modo en que los datos están almacenados para poder construir aplicaciones para su manipulación.
- Imposibilidad de independizar la tarea de los administradores del sistema de la de los desarrolladores de aplicaciones.
- Dificultad para implementar políticas de backup eficientes, puesto que, en caso de necesitar recuperar información de la copia de seguridad, es necesario conservar no sólo los datos, sino la estructura exacta en la que estaban originalmente almacenados.

Con el tiempo, el modelo relacional se ha consolidado como el más utilizado para las aplicaciones de procesamiento de datos, debido a su simplicidad y a su sólida base matemática, que han permitido el desarrollo de lenguajes y herramientas que facilitan el trabajo, tanto de los analistas y diseñadores, como de los programadores, en comparación con otros modelos anteriores.

1.2 Características y conceptos básicos

1.2.1 Definiciones

El modelo relacional se basa en el concepto matemático de **relación**. En este modelo, la información se representa en forma de **tablas** o relaciones, donde cada fila o **tupla** de la tabla se interpreta como una relación ordenada de valores (un conjunto de valores relacionados entre sí)

Véase la siguiente tabla de ejemplo, denominada **profesores**:

NIF	Nombre	Departamento	Teléfono
111111111C	Ana García	Secretaría	100
222222222V	José Gómez	Base de Datos	128
333333333Z	Luisa González	Redes	127
444444444B	Manuel López	Base de Datos	128
555555555P	Antonio Pérez	Dirección	

Formalmente, una relación se define como un conjunto de **tuplas**; donde una tupla se define a su vez como un conjunto ordenado de valores atómicos.

En el ejemplo, la relación mostrada incluye cinco tuplas: ('111111111C', 'Ana García', 'Secretaría', 100), ('222222222V', 'José Gómez', 'Base de Datos', 128), ('333333333Z', 'Luisa González', 'Redes', 127), ('444444444B', 'Manuel López', 'Base de Datos', 128) y ('555555555P', 'Antonio Pérez', 'Dirección',). Cada tupla incluye información sobre profesores: su NIF, su nombre, el departamento al que está adscrito y su número de teléfono. En cada tupla, los cuatro valores están relacionados por el hecho de describir todos ellos al mismo profesor.

Cada relación, vista como una tabla, consta de un conjunto de columnas; cada una de esas columnas recibe el nombre de **atributo**. A cada atributo de una relación le corresponde un nombre, que debe ser único dentro de la relación, y un **dominio**: el conjunto de valores válidos para un atributo; o, dicho de otra manera, el conjunto de valores que cada tupla de la relación puede tomar para ese atributo. A su vez, la unión de todos los dominios de una relación se denomina **universo** de la relación.

En el caso de la relación del ejemplo, los atributos de la misma serían **NIF, Nombre, Departamento y Teléfono**. Cada uno de ellos tendrá un dominio asociado: el conjunto de los NIF válidos (números seguidos de una letra mayúscula calculada de acuerdo con un algoritmo), el conjunto de nombres válidos (cadenas de texto), el conjunto de departamentos existentes (cadenas de texto) y el conjunto de números de teléfono posibles (cadenas de números)

El **esquema** de una relación es una descripción de su estructura interna (es decir, los atributos que la componen), en la forma siguiente:

$$R (A_1, \dots, A_n)$$

siendo R el nombre de la relación, y $A_1, \dots, A_n$ los nombres de sus n atributos. Así, el esquema de la relación **profesores** sería:

$$\text{Profesores} (\text{NIF}, \text{Nombre}, \text{Departamento}, \text{Teléfono})$$

El esquema de una relación constituye su **intensión**, es decir, la parte invariante de la relación. En el ejemplo, el tipo de información que se refleja sobre los profesores será siempre la misma: el nif, el nombre, el departamento al que está adscrito y el número de teléfono. A partir del esquema de la relación surge el concepto de **grado**: el número de atributos de los que consta. La relación del ejemplo sería de grado 4.

Sin embargo, la información recogida en una relación está expuesta constantemente al cambio: la relación de profesores puede sufrir cambios, apareciendo o desapareciendo, o viendo estos modificado su departamento o su teléfono. Se dice que el conjunto de las tuplas que conforman una relación constituye su **extensión**: la parte variable de la relación. De acuerdo con la notación expresada antes, se puede representar a cada tupla de una relación R por medio del siguiente formato:

$$(v_1, \dots, v_n)$$

donde v_1 es el valor de la tupla para el atributo A_1 , y v_n el valor de la tupla para el atributo A_n .

A partir de la información recogida en la relación surge el concepto de **cardinalidad**: el número de tuplas en la relación. En el ejemplo, la cardinalidad sería 5.

En el modelo relacional las tablas presentan además algunas características adicionales:

- Cada fila tiene que ser única, no puede haber filas duplicadas.
- El orden de las filas dentro de una tabla no es relevante.
- El orden de las columnas que forman la tabla no es relevante.
- Los dominios de las distintas columnas no tienen que ser necesariamente disjuntos, pueden no coincidir, coincidir en parte o coincidir completamente.
- Dentro de una tabla, la intersección de una fila y una columna cualesquiera siempre es un dato único.

1.2.2 Valores nulos

En ocasiones, no es posible conocer los valores de algunos atributos para un determinado objeto (para una determinada tupla). Por ejemplo, en el caso de la relación **Profesores** que se está utilizando como referencia, sucede que el profesor con NIF 55555555P no tiene teléfono.

En todos los casos en los que el valor de un atributo para una determinada tupla no se conozca o no aplique por cualquier motivo, el modelo relacional permite el uso de un valor especial, no perteneciente a ningún dominio particular: **el valor nulo**.

NIF	Nombre	Departamento	Teléfono
111111111C	Ana García	Secretaría	100
222222222V	José Gómez	Base de Datos	128

333333333Z	Luisa González	Redes	127
444444444B	Manuel López	Base de Datos	128
555555555P	Antonio Pérez	Dirección	nulo

Es importante señalar que **el valor nulo** no representa realmente un valor, sino la ausencia de valor, por lo que su implementación y tratamiento deben realizarse con precaución. Esta particularidad del **valor nulo** produce los siguientes valores en expresiones lógicas cuando alguno o ambos operandos toman dicho valor:

Op.1	Op.2	Op.1 OR Op.2	Op.1 AND Op.2	Op.1 = Op.2	NOT Op.1	NOT Op.2
True	NULL	True	NULL	NULL	False	NULL
False	NULL	NULL	False	NULL	True	NULL
NULL	True	True	NULL	NULL	NULL	False
NULL	False	NULL	False	NULL	NULL	True
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Más adelante en este documento, cuando se trate el lenguaje SQL, se pondrán algunos ejemplos de este tratamiento especial de los valores nulos.

1.2.3 Restricciones

Para garantizar la consistencia y la facilidad de manipulación de la información representada, existen una serie de reglas que se deben cumplir y que son un elemento constituyente del modelo relacional. A esas reglas se las conoce como **restricciones de integridad**. Se pueden distinguir dos tipos de restricciones: las **inherentes**, que son parte de la propia definición del modelo (la restricción de dominio y la restricción de clave) y las **semánticas**, que ayudan a definir con mayor precisión las características del universo que se pretende modelar (la restricción de identidad de la entidad y la restricción de integridad referencial)

1.2.3.1 Restricción de dominio

Los valores de los atributos de una relación deben ser atómicos.

Esta restricción exige que los valores de cualquier tupla de una relación R correspondientes a los atributos $A_1, \dots, A_n$ de R deben ser valores atómicos. Esto es, esos valores no pueden ser descomponibles en valores más pequeños o simples. Esta condición pretende garantizar que todas las relaciones presenten un formato regular, que pueda ser manipulable por un algoritmo sencillo.

En el caso de la relación **Profesores**, la tupla siguiente no sería válida porque estaría violando la restricción de dominio al tener dos valores en una misma tupla para el atributo **Departamento**.

NIF	Nombre	Departamento	Teléfono
666666666M	Isabel Ramos	Coordinación Secretaría	100

Para representar esta tupla en el modelo relacional, habría que dividirla en dos del modo siguiente:

NIF	Nombre	Departamento	Teléfono
666666666M	Isabel Ramos	Coordinación	100
666666666M	Isabel Ramos	Secretaría	100

1.2.3.2 Restricción de clave

En una relación no puede haber ninguna tupla repetida.

Que una relación no incluya tuplas repetidas, implica que todas las tuplas que contiene puedan ser diferenciadas entre sí por el valor de, al menos, un atributo.

Eso conduce al concepto de **superclave** de una relación: cualquier subconjunto de atributos de la relación, que permite diferenciar a cualesquiera dos tuplas que formen parte de la misma a partir de los valores de las tuplas para esos atributos. Toda relación cuenta con una o más superclaves (en el peor de los casos, existirá una superclave única formada por el conjunto de todos los atributos de la relación). En el caso de la relación **profesores**, con el contenido que se muestra a continuación:

NIF	Nombre	Departamento	Teléfono
111111111C	Ana García	Secretaría	100
222222222V	José Gómez	Base de Datos	128
333333333Z	Luisa González	Redes	127
444444444B	Manuel López	Base de Datos	128
555555555P	Antonio Pérez	Dirección	nulo
666666666M	Isabel Ramos	Coordinación	100
666666666M	Isabel Ramos	Secretaría	100

Serían superclaves los conjuntos de atributos siguientes:

(NIF, Nombre, Departamento, Teléfono)

(NIF, Nombre, Departamento)

(NIF, Departamento, Teléfono)

(Nombre, Departamento, Teléfono)

(NIF, Departamento)

(Nombre, Departamento)

En este caso, por ejemplo, el subconjunto de atributos (Departamento, Teléfono) no puede ser superclave porque tomando sólo esos atributos, existen tuplas repetidas ('Base de Datos', 128). Lo mismo sucedería con el subconjunto de atributos (NIF), puesto que el valor ('666666666M') también estaría en más de una tupla.

Por cuestiones prácticas, lo ideal sería seleccionar un subconjunto lo más pequeño posible de los atributos suficiente para identificar las tuplas. Se denominan **claves candidatas** de la relación todas las superclaves mínimas o no descomponibles, es decir, aquellos conjuntos de atributos de los que ninguno puede ser eliminado sin provocar que el conjunto deje de ser una superclave de la relación. Los atributos que están en alguna clave candidata se denominan **atributos principales**.

En el caso del ejemplo anterior, los únicos conjuntos mínimos los constituirían las superclaves (NIF, Departamento) y (Nombre, Departamento), las otras cuatro son superconjuntos de éstas y por tanto no podrían ser claves candidatas.

Todas las claves candidatas son superclaves mínimas, cuyos valores son suficientes para distinguir a dos tuplas cualesquiera de una relación. A efectos prácticos, el modelo relacional recomienda seleccionar una sola de las posibles claves candidatas para ser utilizada cuando sea necesaria: la escogida será la **clave primaria** de la relación. Cuál sea la elegida queda a discreción del diseñador de la base de datos.

La clave primaria de una relación suele indicarse en la representación del esquema de la misma, subrayando los nombres de los atributos que la componen. Por ejemplo:

Profesores (NIF, Nombre, Departamento, Teléfono)

Las claves constituidas por más de un atributo se denominan **claves compuestas**.

1.2.3.3 Restricción de identidad de la entidad

Ninguna tupla de una relación puede tomar valores nulos en los atributos que forman parte de su clave primaria.

La necesidad de esta restricción viene dada por el hecho de ser la clave primaria la que permite distinguir a las tuplas entre sí, los valores correspondientes a la clave deben ser conocidos en cada tupla para poder diferenciarla.

En el ejemplo, se puede observar que si se admitieran valores nulos en el campo *Departamento*, que forma parte de la clave primaria, podría darse una situación como la siguiente:

NIF	Nombre	Departamento	Teléfono
111111111C	Ana García	Secretaría	100
222222222V	José Gómez	Base de Datos	128
333333333Z	Luisa González	Redes	127
444444444B	Manuel López	Base de Datos	128
555555555P	Antonio Pérez	Dirección	nulo
666666666M	Isabel Ramos	nulo	100
666666666M	Isabel Ramos	nulo	100

En una situación como la mostrada en la tabla anterior, no es posible diferenciar los dos últimos registros entre sí.

1.2.3.4 Restricción de integridad referencial

Si una tupla de una relación R1 hace referencia a una relación R2, debe referirse a una tupla que exista realmente en R2.

Este tipo de restricciones permite garantizar la consistencia en el caso de relaciones que, desde el punto de vista del universo que se esté modelando, mantengan una cierta vinculación.

En el ejemplo que se está empleando, suponiendo que además de la relación **profesores** existiera otra relación llamada **cursos** en la que se recogieran los datos de los distintos cursos impartidos, con la siguiente estructura y contenido:

Código	Nombre	NIF	Departamento
1	SQL básico	222222222V	Base de Datos
2	TCP/IP	333333333Z	Redes
3	Java	777777777W	Programación

En este caso, la relación **cursos** pretende recoger, para cada actividad docente realizada, su código de identificación (representado por el atributo *Código*, que actúa como clave primaria), su nombre y el profesor que la imparte. Para evitar en cada curso repetir toda la información relativa al profesor que lo imparte y que ya está recogida en la relación **profesores**, se incluye únicamente la clave primaria de dicha tabla como referencia, es decir, los atributos *NIF* y *Departamento*.

En los valores mostrados para la relación **cursos**, puede observarse que dos de ellos en efecto están presentes en la relación **profesores**, mientras que el último no lo está, pues no existe ninguna tupla en la misma con los valores 777777777W en el atributo *NIF* y 'Programación' en el atributo

Departamento. Esta tupla estaría violando la restricción de integridad referencial de **cursos** respecto de **profesores**.

Surge de esta manera el concepto de **clave foránea** o **clave ajena**, que se define de la siguiente forma: Dadas dos relaciones R_1 y R_2 y un conjunto de atributos $A_1...A_n$ de R_1 se dice que es clave foránea de R_1 con respecto a R_2 si $A_1...A_n$ es también la clave primaria de R_2 . Dicho de otra manera, $A_1...A_n$ es una clave foránea de R_1 con respecto a R_2 si ese conjunto de atributos figura tanto en el esquema de R_1 como en el de R_2 ; usándose en R_2 como clave primaria, y usándose en R_1 para referenciar a tuplas de R_2 .

1.2.4 Componentes de una base de datos relacional

Una base de datos relacional se compone de:

- Metadatos (también llamados catálogo o diccionario de datos): Son los elementos en la base de datos que describen la propia estructura de la misma.
- Objetos de la base de datos: Tablas, relaciones entre ellas y cualquier otro objeto que se almacena en la base de datos: vistas, índices, secuencias, etc.
- Reglas de integridad: que ayudan a definir el dominio de conocimiento almacenado.

1.3 Reglas de Codd

El éxito que el modelo relacional cosechó desde su publicación condujo a la proliferación de gestores de base de datos que aseguraban ser relacionales. Para poder determinar con certeza si un gestor era fiel al modelo relacional, Codd publicó en 1984 un conjunto de reglas que definen las características esperadas de un sistema relacional, siendo necesario que un gestor cumpla al menos seis de ellas para que pueda considerarse relacional (cuanto mayor sea el número de reglas cumplidas por el gestor, siempre por encima del mínimo de seis, más próximo estará al ideal del modelo)

1.3.1 Regla 0 (regla fundamental)

Para gestionar la base de datos, el sistema gestor debe emplear únicamente sus capacidades relacionales.

1.3.2 Regla 1 (de la información)

Toda la información almacenada en la base de datos debe estar representada como valores en tablas, lo cual incluye, por tanto, la propia información del diccionario de datos.

1.3.3 Regla 2 (del acceso garantizado)

Cualquier información almacenada en la base de datos debe poder ser accesible de manera unívoca mediante un nombre de tabla, un nombre de columna y el valor de la clave primaria para la fila en la que está almacenada la información en cuestión.

1.3.4 Regla 3 (del tratamiento sistemático de los valores nulos)

Los valores nulos se soportan por el gestor de base de datos para representar la falta de información o información desconocida, independientemente del tipo de datos.

1.3.5 Regla 4 (catálogo en línea dinámico basado en el modelo relacional)

A nivel lógico, la descripción de la base de datos se representa de la misma forma que los datos normales y, por tanto, los usuarios autorizados podrán consultar dicha descripción empleando el mismo lenguaje relacional que emplean para consultar los datos.

1.3.6 Regla 5 (del sublenguaje completo de datos)

El sistema gestor puede soportar varios lenguajes y modos de uso de terminal, pero debe existir al menos uno cuyas sentencias sean expresables como cadenas de caracteres, mediante una sintaxis bien definida, y que sea completo, es decir, que permita los siguientes tipos de operaciones:

- Definición de datos.
- Definición de vistas.
- Manipulación de datos (interactiva y por programa)
- Restricciones de integridad.
- Gestión de autorizaciones.
- Limitantes de transacción (inicio-fin, confirmar, deshacer) (begin-end, commit, rollback)

1.3.7 Regla 6 (de actualización de vistas)

El sistema gestor debe ser capaz de actualizar todas las vistas que sean teóricamente actualizables.

1.3.8 Regla 7 (inserción, actualización y borrado de alto nivel)

El sistema gestor debe proporcionar operadores no sólo para recuperar, sino también para insertar, actualizar y borrar conjuntos de datos (de una o más filas y procedentes de una o varias tablas) y no sólo filas una a una.

1.3.9 Regla 8 (independencia física de datos)

Los cambios que puedan producirse en la base de datos a nivel físico (ficheros que almacenan los datos, discos en los que se ubican, etc.) no deben implicar cambios en las aplicaciones que consultan o manipulan los datos, es decir, las aplicaciones son independientes de la infraestructura que soporta la base de datos y el sistema gestor.

Desde el punto de vista de la arquitectura **ANSI/X3/SPARC**, esta regla se refiere a la abstracción que permite que los cambios en el *nivel interno* no supongan tener que modificar el *nivel conceptual*.

1.3.10 Regla 9 (independencia lógica de datos)

Los cambios que puedan producirse en la base de datos a nivel lógico (tablas, filas, columnas) no deben implicar cambios en las aplicaciones que consultan o manipulan los datos. La independencia lógica es más compleja de implementar que la física, siendo las vistas el método más habitual para facilitar esta abstracción.

Desde el punto de vista de la arquitectura **ANSI/X3/SPARC**, esta regla se refiere a la abstracción que permite que los cambios en el *nivel conceptual* no supongan tener que modificar el *nivel externo*.

1.3.11 Regla 10 (independencia de integridad)

Las restricciones de integridad deben poder especificarse en un sublenguaje relacional y almacenarse en el catálogo, no siendo por tanto necesario implementarlas en las aplicaciones que manipulan los datos. Esto permite que los cambios en dichas reglas de integridad no obliguen a modificar las aplicaciones.

1.3.12 Regla 11 (independencia de distribución)

La consulta o manipulación de los datos almacenados debe hacerse de la misma manera independientemente de si la base de datos está centralizada o distribuida. Esta regla implica que el sistema gestor soporta los siguientes tres tipos de transparencia:

- De localización: Los datos se consultan o manipulan siempre igual, sea la base de datos local o remota.

- De fragmentación: Los datos se consultan o manipulan siempre igual, estén las tablas o no fragmentadas.
- De replicación: Los datos se consultan o manipulan siempre igual, estén los datos o no replicados en varias ubicaciones.

1.3.13 Regla 12 (de la no subversión)

Si el sistema gestor proporciona un lenguaje de bajo nivel para manipular los datos, éste no puede permitir saltarse (subvertir) las reglas de integridad definidas sobre la base de datos en lenguajes de más alto nivel.

1.4 Normalización

1.4.1 Conceptos previos

La normalización es un proceso por el que se organizan los atributos y las tablas de una base de datos relacional con el objetivo principal de minimizar la redundancia de datos. Otros beneficios que se obtienen de la normalización son una disminución de problemas potenciales en la actualización de datos y una mejor protección de la integridad de los mismos.

Es importante destacar que la normalización no es una técnica empleada en el diseño de base de datos, sino una etapa posterior al mismo que elimina las dependencias no deseadas entre atributos.

El proceso de normalización de una base de datos se realiza en fases sucesivas, correspondiendo cada una de ellas a lo que se conoce como **forma normal**. Codd definió originalmente tres formas normales (denominadas primera, segunda y tercera forma normal) todas ellas en 1970 y, junto a Raymond F. Boyce una adicional en 1974 conocida como Forma Normal de Boyce-Codd.

Posteriormente, otros autores han definido tres formas normales adicionales (cuarta, quinta y sexta), si bien normalmente (y de hecho la metodología Métrica 3 así lo hace) se considera que una base de datos relacional está suficientemente normalizada si cumple hasta la tercera forma normal.

Para describir las formas normales es necesario introducir los conceptos de dependencia funcional, dependencia funcional completa, dependencia transitiva y dependencia multivaluada.

1.4.1.1 Dependencia funcional

Se dice que un atributo B depende funcionalmente de otro atributo A en la misma relación si a cada valor de A le corresponde un único valor de B. Al atributo A se le denomina **determinante**.

Así, en la siguiente tabla, se puede observar que el atributo **Nombre** depende funcionalmente del atributo **NIF**, mientras que, por ejemplo, el atributo **Departamento** no, puesto que para un mismo valor del atributo **NIF**, hay más de un valor posible para el atributo **Departamento**.

NIF	Nombre	Departamento	Teléfono
111111111C	Ana García	Secretaría	100
222222222V	José Gómez	Base de Datos	128
666666666M	Isabel Ramos	Coordinación	100
666666666M	Isabel Ramos	Secretaría	100

1.4.1.2 Dependencia funcional completa

Se dice que un atributo B depende funcionalmente de un grupo de atributos Z en la misma relación si depende funcionalmente de Z, pero no de ningún subconjunto de Z.

En la siguiente tabla, que almacena información sobre pedidos y artículos incluidos en los mismos, se observa que el atributo **Cantidad** tiene una dependencia funcional completa del grupo de atributos **NumeroPedido** y **Artículo**, al depender funcionalmente de ambos (para un grupo de valores

NumeroPedido-Articulo el atributo **Cantidad** siempre toma el mismo valor), pero no hacerlo de ningún subconjunto (para un mismo valor de **NumeroPedido** o de **Articulo** puede haber distintos valores en el atributo **Cantidad**)

NumeroPedido	Articulo	Cantidad
1000	Bote de Pintura Ref. P123	2
1000	Aguarrás	1
1000	Rodillo Ref. R003	1
1001	Bote de Pintura Ref. P124	2
1001	Aguarrás	2
1001	Rodillo Ref. R003	2
1002	Bote de Pintura Ref. P123	1

1.4.1.3 Dependencia transitiva

Si dados tres atributos (o grupos de atributos) A, B y C de una misma tabla, B depende funcionalmente de A (pero no a la inversa) y C depende funcionalmente de B, se dice que C depende transitivamente de A.

En la siguiente tabla se puede observar que el atributo **CreditosMatricula** depende funcionalmente del atributo **IDMatricula**, que dicha dependencia no se da en sentido inverso y que el atributo **ImporteMatricula** depende funcionalmente del atributo **CreditosMatricula**. Por tanto, se puede decir que el atributo **ImporteMatricula** depende transitivamente del atributo **IDMatricula**.

IDMatricula	Nombre	CreditosMatricula	ImporteMatricula
1000	Ana García	30	900
1001	José Gómez	30	900
1002	Luisa González	24	720
1003	Manuel López	24	720
1004	Antonio Pérez	30	900
1005	Isabel Ramos	36	1080
1006	Isabel Ramos	30	900

1.4.1.4 Dependencia multivaluada

Es una generalización del concepto de dependencia funcional. En este caso, se dice que un atributo A multidetermina a un atributo B si a cada valor de A le corresponde un conjunto definido de valores de B. La tabla del siguiente ejemplo almacena información sobre aeropuertos. Se puede observar que el atributo **Ciudad** multidetermina al atributo **NombreAeropuerto**, puesto que para cada ciudad hay un conjunto perfectamente definido de nombres de aeropuerto

NombreAeropuerto	Ciudad	Inauguración	NumTerminales
Heathrow	Londres	1930	5
Gatwick	Londres	1930	2
Stansted	Londres	1943	1
Adolfo Suárez	Madrid	1931	5

El Prat	Barcelona	1927	2
JFK	Nueva York	1948	8
La Guardia	Nueva York	1934	4

1.4.2 Primera forma normal (1FN)

Se dice que una tabla está en 1FN si no contiene grupos repetitivos, es decir, cada atributo de una tupla tiene a lo sumo un valor. En el ejemplo siguiente se puede ver una tabla que almacena las asignaturas que imparte cada profesor y el departamento al que dicho profesor está adscrito y que no está en 1FN:

Profesor	Asignatura	Departamento
Ana García	Cálculo	Matemáticas
José Gómez	Electrónica	Ciencias
Luisa González	Cálculo Álgebra	Matemáticas
Manuel López	Electrónica	Ciencias

No está en 1FN porque el atributo **Asignatura**, en la tercera tupla tiene dos valores. La tabla una vez normalizada para cumplir con la 1FN, quedaría del siguiente modo:

Profesor	Asignatura	Departamento
Ana García	Cálculo	Matemáticas
José Gómez	Electrónica	Ciencias
Luisa González	Cálculo	Matemáticas
Luisa González	Álgebra	Matemáticas
Manuel López	Electrónica	Ciencias

1.4.3 Segunda forma normal (2FN)

Se dice que una tabla está en 2FN si está en 1FN y además todos los atributos no principales (es decir, que no formen parte de ninguna clave candidata) tienen dependencia funcional completa respecto de las claves candidatas.

Retomando el ejemplo del apartado anterior, en la tabla que quedó normalizada en 1FN, se puede observar que la única clave candidata posible es la formada por los atributos **Profesor** y **Asignatura**. También puede observarse que el único atributo no principal de la tabla, **Departamento**, no tiene una dependencia funcional completa de la clave, puesto que depende funcionalmente de un subconjunto de la misma, en este caso el atributo **Profesor**. Por tanto, no está en 2FN.

Para normalizar esta tabla, habría que descomponerla en dos, por ejemplo, de la siguiente forma:

Profesor	Asignatura
Ana García	Cálculo
José Gómez	Electrónica
Luisa González	Cálculo

Luisa González	Álgebra
Manuel López	Electrónica

Profesor	Departamento
Ana García	Matemáticas
José Gómez	Ciencias
Luisa González	Matemáticas
Manuel López	Ciencias

1.4.4 Tercera forma normal (3FN)

Se dice que una tabla está en 3FN si está en 2FN y además ningún atributo no principal depende transitivamente de la clave (es decir, todos los atributos de la tabla dependen sólo de la clave principal). En el ejemplo siguiente se puede ver una tabla que almacena el nombre del coordinador de cada asignatura durante un curso lectivo, así como su fecha de ingreso y que no cumple con la 3FN.

Asignatura	Curso	Coordinador	FechaIngreso
Cálculo	2013-2014	Ana García	15-Sept-2005
Cálculo	2014-2015	Luisa González	15-Sept-2008
Álgebra	2014-2015	Ana García	15-Sept-2005
Electrónica	2014-2015	Manuel López	15-Sept-2013

El motivo es que, en este caso, la clave principal de la tabla la forman los atributos **Asignatura** y **Curso**, sin embargo, el atributo **FechaIngreso** depende del atributo **Coordinador**, que no es la clave principal. Para normalizar esta tabla y que sea conforme con la 3FN, la tabla debe descomponerse en dos, siendo el atributo no principal del que existe alguna dependencia el campo que actúa como enlace entre ambas, como se muestra a continuación.

Asignatura	Curso	Coordinador
Cálculo	2013-2014	Ana García
Cálculo	2014-2015	Luisa González
Álgebra	2014-2015	Ana García
Electrónica	2014-2015	Manuel López

Coordinador	FechaIngreso
Ana García	15-Sept-2005
Luisa González	15-Sept-2008
Manuel López	15-Sept-2013

1.4.5 Forma normal de Boyce-Codd (FNBC)

Es una forma normal muy similar a la 3FN, pero ligeramente más restrictiva que ayuda a modelar de una manera más adecuada ciertas restricciones semánticas. Una tabla está en FNBC si está en 3FN y además todos los atributos que son determinantes son claves candidatas.

El ejemplo siguiente muestra una tabla de profesores, con el departamento al que pertenecen y el coordinador del mismo (un profesor puede pertenecer a más de un departamento). Esta tabla está en 3FN y su clave primaria la forman los atributos **Profesor** y **Departamento**.

Profesor	Departamento	Coordinador
Ana García	Matemáticas	Ana García
José Gómez	Matemáticas	Ana García
Luisa González	Ciencias	Manuel López
Manuel López	Ciencias	Manuel López
Ángel Sánchez	Matemáticas	Ana García
Ángel Sánchez	Ciencias	Manuel López

Si ahora se añade una nueva restricción semántica, que no es posible deducir simplemente observando la tabla, que indique que un profesor sólo puede ser coordinador de una única asignatura, esto supondría la aparición de una nueva dependencia funcional entre **Departamento** y **Coordinador** y la tabla del ejemplo, que a la vista de los valores que contiene está en 3FN, no está en cambio en la FNBC, puesto que el atributo **Coordinador** es determinante sin ser clave candidata.

La forma de normalizar esta situación para que la tabla sea conforme a la FNBC es, como sucedía en el caso anterior, dividir la tabla en dos que se relacionen entre sí por el atributo que depende funcionalmente de un atributo no principal. En este caso:

Profesor	Departamento
Ana García	Matemáticas
José Gómez	Matemáticas
Luisa González	Ciencias
Manuel López	Ciencias
Ángel Sánchez	Matemáticas
Ángel Sánchez	Ciencias

Departamento	Coordinador
Matemáticas	Ana García
Ciencias	Manuel López

1.4.6 Cuarta forma normal (4FN)

Una tabla está en 4FN si está en 3FN y además toda dependencia multivaluada no trivial está determinada por una clave candidata. En el siguiente ejemplo se muestra una tabla que cumple con 1FN, 2FN, 3FN y FNBC y almacena los datos de los profesores y las asignaturas que imparten por curso. La clave primaria la forman los tres atributos de la tabla.

Asignatura	Profesor	Curso
Cálculo	Ana García	Primero
Cálculo	Ana García	Segundo
Cálculo	Luisa González	Primero
Cálculo	Luisa González	Segundo

Álgebra	Luisa González	Primero
Álgebra	Luisa González	Segundo
Electrónica	José Gómez	Segundo
Electrónica	Manuel López	Segundo

Se puede observar que hay dos dependencias multivaluadas. El atributo **Asignatura** multidetermina al atributo **Profesor** y también al atributo **Curso**. Dado que los atributos **Profesor** y **Curso** no tienen ninguna dependencia entre ellos (no existe ninguna relación entre los profesores que imparten una asignatura y los cursos donde dicha asignatura se imparte) se está introduciendo una redundancia de información en la tabla. La 4FN soluciona este problema dividiendo la tabla anterior en dos, como se muestra a continuación, de manera que las dependencias multivaluadas se almacenan de manera correcta evitando redundancias.

Asignatura	Profesor
Cálculo	Ana García
Cálculo	Luisa González
Álgebra	Luisa González
Electrónica	José Gómez
Electrónica	Manuel López

Asignatura	Curso
Cálculo	Primero
Cálculo	Segundo
Álgebra	Primero
Álgebra	Segundo
Electrónica	Segundo

1.4.7 Quinta forma normal (5FN)

La 5FN se emplea para facilitar el mantenimiento de determinados esquemas de datos complejos, permitiendo garantizar la integridad de los datos. Es bastante compleja de entender y gestionar y no se suele emplear, quedando reservada a casos en los que hay esquemas de datos con tablas con mucha redundancia de datos y pocos atributos, o tablas con muchos atributos que se dividen en varias tablas para hacer más fácil su visualización y gestión.

Retomando el ejemplo que se mostraba en el caso de la 4FN. Supóngase ahora que aparece un nuevo profesor, José Sánchez, que va a dar clases de Cálculo, pero sólo en el primer curso. El modelo de datos actual, a través únicamente de las restricciones implementadas en la base de datos no permite expresar esta condición. Se insertaría un solo registro en base de datos, pero un vistazo a su contenido, aunque permitiría saber que José Sánchez sólo da clase en el primer curso, no permitiría distinguir si eso se debe a una restricción puramente circunstancial o a una restricción firme del universo que se pretende modelar.

Asignatura	Profesor	Curso
Cálculo	Ana García	Primero
Cálculo	Ana García	Segundo

Cálculo	Luisa González	Primero
Cálculo	Luisa González	Segundo
Álgebra	Luisa González	Primero
Álgebra	Luisa González	Segundo
Electrónica	José Gómez	Segundo
Electrónica	Manuel López	Segundo
Cálculo	José Sánchez	Primero

Para implementar claramente esta restricción y que esta tabla esté en 5FN, debería dividirse en tres tablas diferentes, que expresaran respectivamente la relación entre asignaturas y profesores, entre asignaturas y cursos y entre cursos y profesores.

Asignatura	Profesor
Cálculo	Ana García
Cálculo	Luisa González
Cálculo	José Sánchez
Álgebra	Luisa González
Electrónica	José Gómez
Electrónica	Manuel López

Asignatura	Curso
Cálculo	Primero
Cálculo	Segundo
Álgebra	Primero
Álgebra	Segundo
Electrónica	Segundo

Curso	Profesor
Primero	Ana García
Primero	Luisa González
Primero	José Sánchez
Segundo	Ana García
Segundo	Luisa González
Segundo	José Gómez
Segundo	Manuel López

1.4.8 Sexta forma normal (6FN)

Igual que sucedía con la 5FN, es bastante compleja de entender y gestionar y no se suele emplear. A efectos prácticos podría decirse que una tabla está en 6FN si no puede descomponerse en tablas más pequeñas sin pérdida de información. La 6FN está pensada para gestionar información temporal

(que varía a lo largo del tiempo) en bases de datos, minimizando el número de operaciones necesario para mantener ese tipo de información.

El siguiente ejemplo muestra una tabla de empleados, con su NIF, localidad de residencia, periodo de tiempo en el que la información es válida, departamento al que pertenece y estado (alta, baja)

Supóngase que el empleado 111111111C, residente en Madrid se da de alta con un contrato de un año durante 2001 para trabajar en el departamento de logística. El registro que se insertaría sería el siguiente:

NIF	Localidad	FechaDesde	FechaHasta	Departamento	Estado
111111111C	Madrid	01-01-2001	31-12-2001	Logística	Alta

Si ahora el empleado se da de baja por enfermedad durante la primera semana de mayo, para mantener la información actualizada sería preciso insertar dos nuevos registros, duplicándose innecesariamente buena parte de la información:

NIF	Localidad	FechaDesde	FechaHasta	Departamento	Estado
111111111C	Madrid	01-01-2001	30-04-2001	Logística	Alta
111111111C	Madrid	01-05-2001	07-05-2001	Logística	Baja
111111111C	Madrid	08-05-2001	31-12-2001	Logística	Alta

Como se puede ver, el problema deriva del hecho de que la información está sujeta a periodos de validez, determinados por el valor de los campos FechaDesde y FechaHasta. La 6FN propondría en este caso descomponer la información en varias tablas, de manera que se minimice el impacto en el mantenimiento de la tabla que tendría la aparición de una nueva situación con un periodo de validez

Así, deberían crearse cuatro tablas, que en la situación inicial serían las siguientes (la información almacenada en cada una sería respectivamente: periodo durante el cual el empleado tiene relación con la empresa, periodo en el que el empleado ha tenido su residencia en un determinado lugar, periodo en el que el empleado ha trabajado en determinado departamento y periodo en el que el empleado ha tenido determinado estado en su relación con la empresa):

NIF	FechaDesde	FechaHasta
111111111C	01-01-2001	31-12-2001

NIF	Localidad	FechaDesde	FechaHasta
111111111C	Madrid	01-01-2001	31-12-2001

NIF	FechaDesde	FechaHasta	Departamento
111111111C	01-01-2001	31-12-2001	Logística

NIF	FechaDesde	FechaHasta	Estado
111111111C	01-01-2001	31-12-2001	Alta

Así, si el empleado con este esquema de tablas se da de baja durante la primera semana de mayo, sólo es necesario insertar registros en la cuarta tabla, minimizándose la información a duplicar.

NIF	FechaDesde	FechaHasta	Estado
111111111C	01-01-2001	30-04-2001	Alta
111111111C	01-05-2001	07-05-2001	Baja
111111111C	08-05-2001	31-12-2001	Alta

1.5 Base matemática del modelo relacional. Lenguajes relacionales

Como se comentó anteriormente, una de las claves del éxito del modelo de datos relacional, además de su sencillez y potencia a la hora de permitir el modelado de situaciones del mundo real, es la potente base matemática en la que se apoya.

Dicha base matemática, además de permitir garantizar la consistencia del modelado, posibilita la creación de lenguajes de consulta de base de datos basados en principios matemáticos y, por tanto, con plena garantía de que la recuperación de datos se realiza correctamente (esto es, se recuperar toda la información deseada y sólo la información deseada)

Los formalismos matemáticos en los que se apoya el modelo relacional son fundamentalmente el álgebra relacional y el cálculo relacional de tuplas, en los que se basa el lenguaje SQL. Existen otros lenguajes de consulta de base de datos menos populares, como QBE, basados en un tercer formalismo, el cálculo relacional de predicados, que no se tratará en este documento.

1.5.1 Álgebra relacional

El álgebra relacional es un lenguaje de consulta procedimental que consta de un conjunto de operaciones que toman como entrada una o dos relaciones y producen como resultado una nueva relación.

Las operaciones fundamentales, también llamadas operadores básicos, del álgebra relacional son *selección*, *proyección*, *unión*, *diferencia de conjuntos*, *producto cartesiano* y *renombramiento*. Estas operaciones son suficientes para expresar cualquier consulta en álgebra relacional.

A lo largo de los siguientes puntos se describen estas operaciones. Los ejemplos mostrados en cada caso usan siempre las siguientes dos tablas:

profesores

NIF	Nombre	Departamento	Teléfono
222222222V	José Gómez	Base de Datos	128
333333333Z	Luisa González	Redes	127
444444444B	Manuel López	Base de Datos	128

catedráticos

NIF	Nombre	FechaNacimiento
222222222V	José Gómez	01-01-1960
123P	Joaquín Jiménez	01-02-1961

1.5.1.1 Selección (σ)

Es una operación que selecciona las tuplas que satisfacen un determinado predicado. Se representa con la letra griega sigma (σ). La operación

$\sigma_{\text{telefono}=128}(\text{profesores})$ daría como resultado;

NIF	Nombre	Departamento	Teléfono
-----	--------	--------------	----------

222222222V	José Gómez	Base de Datos	128
444444444B	Manuel López	Base de Datos	128

1.5.1.2 Proyección (Π)

Es una operación que devuelve un subconjunto de los atributos de una relación. Se representa con la letra griega pi (Π). La operación

$\Pi_{\text{nif, nombre}}(\text{profesores})$ daría como resultado:

NIF	Nombre
222222222V	José Gómez
333333333Z	Luisa González
444444444B	Manuel López

1.5.1.3 Unión ($\cup$)

Es una operación que devuelve una relación con las tuplas que están en dos relaciones, eliminando las tuplas duplicadas. Se denota, igual que en teoría de conjuntos, con el símbolo $\cup$. La operación

$\Pi_{\text{nif, nombre}}(\text{profesores}) \cup \Pi_{\text{nif, nombre}}(\text{catedraticos})$ daría como resultado:

NIF	Nombre
222222222V	José Gómez
333333333Z	Luisa González
444444444B	Manuel López
123P	Joaquín Jiménez

En este caso, es preciso señalar que para que la operación de unión sea posible las dos relaciones que se pretende unir deben ser **compatibles**. Para ello se deben dar dos condiciones:

- Las relaciones deben tener el mismo número de atributos.
- Los dominios de los atributos *i-ésimos* de ambas relaciones deben ser iguales para todo *i*.

1.5.1.4 Diferencia de conjuntos (-)

Es una operación que devuelve una relación con las tuplas que están en una relación pero no en otra. Como en el caso de la unión, las dos relaciones implicadas en la operación han de ser compatibles. Se denota con el símbolo $-$. La operación:

$\Pi_{\text{nif, nombre}}(\text{profesores}) - \Pi_{\text{nif, nombre}}(\text{catedraticos})$ daría como resultado:

NIF	Nombre
333333333Z	Luisa González
444444444B	Manuel López

1.5.1.5 Producto cartesiano ($\times$)

Es una operación entre dos relaciones cuyo resultado es la combinación de todas las tuplas de la primera relación con todas las tuplas de la segunda. Se denota con el símbolo $\times$. La operación

$\Pi_{\text{nif}}(\text{profesores}) \times \Pi_{\text{nif}}(\text{catedraticos})$ daría como resultado:

NIF	NIF2
22222222V	123P
33333333Z	123P
44444444B	123P
22222222V	22222222V
33333333Z	22222222V
44444444B	22222222V

1.5.1.6 Renombramiento (ρ)

Es una operación que, dada una expresión, devuelve el resultado de la misma con un nombre determinado. Se denota con la letra griega rho (ρ).

Esta operación se justifica por el hecho de que en álgebra relacional el resultado de una operación cualquiera devuelve siempre una relación anónima. Para poder referirse a dicha relación anónima de alguna manera se emplea la operación de **renombramiento**. La operación

$\rho_{\text{cruce}} (\Pi_{\text{nif}} (\text{profesores}) \times \Pi_{\text{nif}} (\text{catedraticos}))$ daría el mismo resultado indicado anteriormente, dándole el nombre **cruce**. Dado que dicha relación resultado de la operación tiene un nombre, podría referenciarse en operaciones posteriores, por ejemplo

$\sigma_{\text{nif}=22222222V} (\text{cruce})$ daría como resultado:

NIF	NIF2
22222222V	123P
22222222V	22222222V

1.5.1.7 Otras operaciones adicionales

Existen otras operaciones que se definen en términos de las operaciones fundamentales. Su definición se justifica porque, aunque con las operaciones fundamentales puede construirse cualquier consulta, las expresiones que resultan en ocasiones son demasiado intrincadas. Gracias a estas operaciones adicionales, también llamadas operadores derivados, éstas resultan más legibles y sencillas de entender.

1.5.1.7.1 Intersección

Dadas dos relaciones, la intersección, que se denota con el mismo símbolo que en teoría de conjuntos ($\cap$), devuelve como resultado las tuplas que están en ambas relaciones. Las dos relaciones implicadas deben ser compatibles. La operación

$\Pi_{\text{nif, nombre}} (\text{profesores}) \cap \Pi_{\text{nif, nombre}} (\text{catedraticos})$ daría como resultado:

NIF	Nombre
22222222V	José Gómez

La operación intersección, equivale a la siguiente combinación de operaciones fundamentales:

$$r \cap s = r - (r - s)$$

1.5.1.7.2 Unión natural

Dadas dos relaciones, la operación unión natural, devuelve las tuplas que, en el producto cartesiano de ambas, cumplen una determinada condición. Esta operación se denota con el símbolo $\bowtie$

Así, la operación

$\text{profesores} \bowtie_{\text{nif2}=123P} \text{catedráticos}$ daría como resultado:

NIF	NIF2
222222222V	123P
333333333Z	123P
444444444B	123P

La operación unión natural, equivale a la siguiente combinación de operaciones fundamentales:

$$r \bowtie_p s = \sigma_p (r \times s)$$

Esta operación, cuando la condición que se aplica es que se igualen los atributos coincidentes en ambas relaciones, es la que se emplea habitualmente para unir dos tablas relacionadas entre sí mediante una clave foránea. En ocasiones dicha operación también se conoce como unión o join, y se denota mediante el símbolo *. Así, la operación

profesores * catedraticos
 es equivalente a
 profesores $\bowtie_{\text{profesores.nif=catedraticos.nif}}$ catedraticos
 que a su vez es equivalente a
 $\sigma_{\text{profesores.nif=catedraticos.nif}}$ (profesores X catedraticos)
 y que daría como resultado:

NIF	Nombre	Departamento	Teléfono	NIF2	Nombre2	FechaNacimiento
222222222V	José Gómez	Base de Datos	128	222222222V	José Gómez	01-01-1960

Outer join

Es una operación derivada de la unión natural que se considera una extensión del álgebra relacional. En este caso, la operación tiene tres variantes: izquierda, derecha y completa.

- Left outer join: Equivale a realizar la unión natural de las dos tablas implicadas, añadiendo a continuación al resultado todas las tuplas que están en la primera de las dos tablas para las que no aparezcan registros asociados a ellas en la segunda. La operación

profesores *_{LEFT} catedraticos
 da como resultado

NIF	Nombre	Departamento	Teléfono	NIF2	Nombre2	FechaNacimiento
222222222V	José Gómez	Base de Datos	128	222222222V	José Gómez	01-01-1960
333333333Z	Luisa González	Redes	127	null	null	null
444444444B	Manuel López	Base de Datos	128	null	null	null

- Right outer join: Equivale a realizar la unión natural de las dos tablas implicadas, añadiendo a continuación al resultado todas las tuplas que están en la segunda de las dos tablas para las que no aparezcan registros asociados a ellas en la primera. La operación

profesores *_{RIGHT} catedraticos
 da como resultado

NIF	Nombre	Departamento	Teléfono	NIF2	Nombre2	FechaNacimiento
222222222V	José Gómez	Base de Datos	128	222222222V	José Gómez	01-01-1960
null	null	null	null	123P	Joaquín Jiménez	01-02-1961

- Full outer join: Equivale a realizar la unión natural de las dos tablas implicadas, añadiendo a continuación al resultado tanto las tuplas que están en la primera de las dos tablas para las que no aparezcan registros asociados a ellas en la segunda, como las tuplas que están en la segunda de las dos tablas para las que no aparezcan registros asociados a ellas en la primera. La operación

profesores *_{FULL} catedraticos
da como resultado

NIF	Nombre	Departamento	Teléfono	NIF2	Nombre2	FechaNacimiento
222222222V	José Gómez	Base de Datos	128	222222222V	José Gómez	01-01-1960
333333333Z	Luisa González	Redes	127	null	null	null
444444444B	Manuel López	Base de Datos	128	null	null	null
null	null	null	null	123P	Joaquín Jiménez	01-02-1961

1.5.1.7.3 División o cociente

Dadas dos tablas, R y S, considerando en la primera un grupo de atributos (x) y un atributo (y) y en la segunda un atributo (y), la división, denotada por el símbolo /, devolverá aquellos valores del grupo de atributos (x) existentes en la tabla R para los que existe en R un par (x,y), para todos los valores de (y) existentes en la tabla S. El siguiente ejemplo muestra la operación división:

Sea R:

NIF	Clave
222222222V	1
222222222V	2
222222222V	3
333333333Z	2
333333333Z	3
444444444B	1
444444444B	2

Y sea S:

Clave
1
3

El resultado de la operación R / S será:

NIF
222222222V

El cálculo de este resultado mediante operaciones fundamentales sería el siguiente:

- Calcular $\rho_{X1} (\Pi_{nif} (R))$, cuyo resultado es:

NIF
222222222V
333333333Z
444444444B

- Calcular $\rho_{Y1} (S \times X1)$, cuyo resultado es:

NIF	Clave
222222222V	1
222222222V	3
333333333Z	1
333333333Z	3
444444444B	1
444444444B	3

- Calcular $\rho_{Y2} (Y1 - R)$, cuyo resultado es:

NIF	Clave
333333333Z	1
444444444B	3

- Calcular $\rho_{X2} (\Pi_{nif} (Y2))$, cuyo resultado es:

NIF
333333333Z
444444444B

- Calcular $X1 - X2$, cuyo resultado es:

NIF
222222222V

Esta operación es la única para la que el estándar SQL no contempla un operador específico.

1.5.2 Cálculo relacional de tuplas

El cálculo relacional de tuplas es un lenguaje de consulta declarativo que permite construir expresiones equivalentes a las del álgebra relacional. En los siguientes apartados se mostrará el equivalente en cálculo relacional de tuplas de las operaciones fundamentales del álgebra relacional.

1.5.2.1 Selección

La operación que en álgebra relacional toma la forma:

$\sigma_{\text{telefono}=128} (\text{profesores})$

equivale en cálculo relacional de tuplas a la expresión:

$\{p / p \in \text{profesores} \wedge p.\text{telefono}=128\}$

1.5.2.2 Proyección

La operación que en álgebra relacional toma la forma:

$$\Pi_{\text{nif, nombre}}(\text{profesores})$$

equivale en cálculo relacional de tuplas a la expresión:

$$\{p[\text{nif, nombre}] / p \in \text{profesores}\}$$

1.5.2.3 Unión

La operación que en álgebra relacional toma la forma:

$$\Pi_{\text{nif, nombre}}(\text{profesores}) \cup \Pi_{\text{nif, nombre}}(\text{catedraticos})$$

equivale en cálculo relacional de tuplas a la expresión:

$$\{p[\text{nif, nombre}] / p \in \text{profesores} \vee p \in \text{catedraticos}\}$$

1.5.2.4 Diferencia de conjuntos

La operación que en álgebra relacional toma la forma:

$$\Pi_{\text{nif, nombre}}(\text{profesores}) - \Pi_{\text{nif, nombre}}(\text{catedraticos}) \text{ daría como resultado:}$$

equivale en cálculo relacional de tuplas a la expresión:

$$\{p[\text{nif, nombre}] / p \in \text{profesores} \wedge \neg \exists c \in \text{catedraticos} / (p.\text{nif}=c.\text{nif} \wedge p.\text{nombre}=c.\text{nombre})\}$$

1.5.2.5 Producto cartesiano

La operación que en álgebra relacional toma la forma:

$$\Pi_{\text{nif}}(\text{profesores}) \times \Pi_{\text{nif}}(\text{catedraticos}) \text{ daría como resultado:}$$

equivale en cálculo relacional de tuplas a la expresión:

$$\{p.\text{nif}, c.\text{nif} \vee p \in \text{profesores}, \forall c \in \text{catedraticos}\}$$

1.6 Conclusiones

De cara al estudio de este apartado del tema es conveniente tener claros los conceptos que se introducen en los primeros puntos (resaltados en negrita), así como las características de las tablas en el modelo relacional. Es importante entender el concepto de valor nulo y su funcionamiento, así como el significado de cada uno de los cuatro tipos de restricciones y conocer los componentes de una base de datos relacional. Las primeras preguntas de test que se incluyen al final del documento pueden dar una idea del tipo de pregunta que podría hacerse en un test y que no busca un conocimiento de pura memoria, sino de comprensión del tema.

Deben conocerse las reglas de Codd, aunque no debería ser necesario saber su orden ni nombre exacto, bastaría con entender lo que cada una aporta al modelo.

Deben entenderse perfectamente las tres primeras formas normales que son las exigidas por Métrica, puesto que es frecuente que aparezcan preguntas relacionadas con ellas en el test (fabricarse ejemplos propios es un buen ejercicio) y conocer las demás (aunque es poco probable que en un test se pueda hacer una pregunta sobre ellas que vaya más allá de la pura teoría, dada su complejidad)

Es también importante conocer la sintaxis y entender las operaciones del álgebra relacional, tanto las fundamentales como las adicionales, puesto que en este caso sí es fácil construir preguntas de test relacionadas con ellas (en el examen de 2009, la segunda pregunta del test era una operación división). Para determinar el grado de dominio que se tiene sobre estas operaciones puede ser muy útil disponer de la posibilidad de acceder a una base de datos sobre la que poder probar las diferentes operaciones (en este caso, a través de su correspondiente operador SQL). En el caso el cálculo relacional, es suficiente con entender la correspondencia de las operaciones con las del álgebra relacional y conocer su sintaxis.

2 El lenguaje SQL

2.1 Introducción y breve referencia histórica

SQL (Structured Query Language) es un lenguaje de propósito específico diseñado para la gestión de los datos almacenados en sistemas de bases de datos relacionales. Su definición original se basa en el álgebra relacional y el cálculo relacional de tuplas e incluía sentencias, tanto para la definición de datos (DDL) como para la manipulación de los mismos (DML). Incluye por tanto instrucciones para la inserción, borrado, actualización y consulta de información, creación y modificación de esquemas de base de datos y gestión de permisos para realizar todas estas operaciones.

Es en la actualidad el lenguaje más empleado en el manejo de bases de datos relacionales, habiéndose convertido en un estándar ANSI en 1986 y en un estándar ISO un año más tarde. A lo largo de los años estos estándares han ido evolucionando para dotar al lenguaje de mayor potencia y nuevas funcionalidades. A pesar de su estandarización, parte del código SQL no es completamente transportable entre bases de datos de distintos proveedores sin la realización de ciertos ajustes.

SQL se desarrolla por parte de Donald Chamberlin y Raymond Boyce para IBM en 1970. Su propósito inicial es la gestión de un sistema de base de datos cuasi-relacional, el sistema R de IBM. Su nombre original era SEQUEL, pero por problemas con la propiedad de dicho nombre se abrevió hasta SQL.

A finales de 1979 la compañía que sería el embrión de la futura Oracle presenta la primera implementación comercial de SQL, Oracle V2, disponible para sistemas VAX. Por su parte, IBM, también a partir de SQL, desarrolla productos que culminan en la aparición en 1983 del sistema DB2.

2.2 Estandarización y versiones

Como se comentaba en el punto anterior, en 1986 ANSI adopta SQL como estándar y un año más tarde lo hace ISO. Actualmente la evolución del estándar la realiza el Comité Técnico Conjunto (Joint Technical Committee) ISO/IEC JTC-1 (Tecnologías de la Información) y dentro de éste, por el Subcomité SC 32 (Intercambio y Gestión de Datos). El número de estándar es ISO 9075.

Lo complementa el estándar ISO 13249: Paquetes SQL Multimedia y Paquetes de Aplicación, que define interfaces y paquetes basados en SQL para soporte de video, audio y georreferenciación.

La lista de revisiones realizadas al estándar desde 1986 se muestra a continuación:

Año	Nombre	Aportaciones fundamentales
1986	SQL-86	Primera normalización realizada por ANSI
1989	SQL-89	Incorporación de reglas de identidad referencial
1992	SQL-92	La revisión más importante y que ya contiene la mayoría de las funcionalidades más importantes del lenguaje. ISO 9075
1999	SQL-1999	Soporte a expresiones regulares, consultas recursivas, triggers, tipos de datos no escalares y ciertas características de orientación a objetos. Soporte para embeber SQL en Java y viceversa.
2003	SQL-2003	Funcionalidades XML, secuencias estandarizadas, columnas con valores autogenerados.
2006	SQL-2006	Uso de SQL junto a XML, almacenamiento, manipulación y presentación de XML en bases de datos SQL. Soporte de XQuery
2008	SQL-2008	Uso de ORDER BY fuera de la definición de cursores. Triggers <i>INSTEAD OF</i> . Sentencia <i>TRUNCATE</i>
2011	SQL-2011	Soporte para bases de datos temporales

2.3 Elementos del lenguaje

2.3.1 Tipos de datos

El estándar SQL soporta múltiples tipos de datos, si bien los distintos fabricantes de bases de datos soportan diferentes tipos adicionales. Estos tipos de datos pueden agruparse en cuatro grandes grupos:

- Cadenas de caracteres: Incluye los tipos:
 - CHARACTER (n) o CHAR (n): cadenas de caracteres de tamaño fijo (n) rellenas con el número de espacios necesario hasta llegar a ese número n.
 - CHARACTER VARYING (n) o VARCHAR (n): cadenas de caracteres de tamaño variable hasta un máximo de n.
 - NATIONAL CHARACTER (n) o NCHAR (n): similar a CHAR (n) con capacidad para dar soporte a alfabetización internacional.
 - NATIONAL CHARACTER VARYING (n) o NVARCHAR (n): similar a VARCHAR (n) con capacidad para dar soporte a alfabetización internacional.
- Cadenas de bits:
 - BIT (n): cadenas de n bits.
 - BIT VARYING (n): cadenas con un máximo de n bits.
- Números:
 - INTEGER, SMALLINT, BIGINT: para números enteros, de distinto tamaño máximo.
 - FLOAT, REAL, DOUBLE PRECISION: para números en coma flotante, con distintos tamaños para la parte entera y la parte decimal.
 - NUMERIC (dígitos, decimales), DECIMAL (dígitos, decimales): números decimales con la precisión deseada. El parámetro *dígitos* indica el total de dígitos disponibles, incluyendo tanto la parte entera como la decimal; el parámetro *decimales* indica el número de decimales.
- Fecha y hora:
 - DATE: Para fechas.
 - TIME: Para horas. Normalmente se llega a una precisión de 100 nanosegundos.
 - TIME WITH TIME ZONE, TIMETZ: Igual que TIME, pero incluyendo información sobre zona horaria.
 - TIMESTAMP: Fecha y hora en un solo dato.
 - TIMESTAMP WITH TIME ZONE, TIMESTAMPTZ: Igual que TIMESTAMP, pero incluyendo información sobre zona horaria.

El estándar incluye además dos funciones para redondeo de números, ROUND (redondeo) y TRUNC (truncado, esto es, eliminación de los decimales) y numerosas funciones para manipulación de fechas (conversión de cadenas de caracteres en fechas y viceversa, selección de partes de una fecha o una hora, etc.)

2.3.2 Operadores y expresiones condicionales

Los operadores contemplados por el estándar SQL son los siguientes:

=	->	igual a (a = 3)
<>	->	distinto de (a <> 3; también se suele aceptar a !=3)
>	->	mayor que (a > 3)
<	->	menor que (a < 3)
>=	->	mayor o igual que (a >= 3)
<=	->	menor o igual que (a <= 3)
BETWEEN...AND	->	entre dos valores (a BETWEEN 3 AND 5)
LIKE	->	similar a un patron (a LIKE '%rueba%')
IN	->	igual a uno de un conjunto de valores (a IN (3, 4, 5))
IS, IS NOT	->	comparación con el valor nulo (a IS NULL; a IS NOT NULL)

Contempla además expresiones condicionales construidas con la palabra clave CASE; se muestran dos ejemplos a continuación:

```

CASE a WHEN 1
            THEN 'uno'
        WHEN 2
            THEN 'dos'
        ELSE
            'mayor de dos'
END

CASE  WHEN a > 0
            THEN 'Es positivo'
        WHEN a < 0
            THEN 'Es negativo'
        ELSE 'Es cero'
END
    
```

2.3.3 Consultas (Queries)

La sentencia fundamental para hacer consultas en una base de datos SQL es SELECT. Se muestra a continuación su sintaxis básica y se describen a continuación sus distintos componentes, acompañados de diversos ejemplos.

```

SELECT [ALL | DISTINCT ]
        <lista de campos y/o funciones agregadas a mostrar>
FROM <lista de tablas y/o vistas a consultar>
[WHERE <lista de condiciones para la búsqueda>]
[GROUP BY <lista de campos para agrupar>]
[HAVING <lista de condiciones para las funciones agrupadas>]
[ORDER BY <lista de campos para ordenar> [ASC | DESC]]
    
```

Las únicas cláusulas obligatorias son **SELECT**, que va seguido de la lista de campos y/o funciones agregadas a recuperar y **FROM**, que va seguido de la lista de tablas y/o vistas de las que recuperar la información. Se muestra a continuación una consulta básica con estas dos cláusulas:

```

SELECT nif, nombre_completo, dirección, teléfono, id_departamento FROM profesores
    
```

Las funciones agregadas se emplean para agrupar los registros consultados y ofrecer datos sobre el conjunto de ellos en lugar de obtenerlos uno a uno de manera individual. Son cinco las contempladas en el estándar: **COUNT** (cuenta los registros devueltos por la consulta), **SUM** (suma los valores de un campo determinado para los registros recuperados en la consulta), **AVG** (hace la media de los valores de un campo determinado para los registros recuperados en la consulta), **MAX** (devuelve el valor máximo de un campo determinado para los registros recuperados en la consulta), **MIN** (ídem para el valor mínimo). El ejemplo mostrado a continuación no recupera información particular sobre registros individuales de la tabla de profesores, sino que devuelve el total de registros que hay en la misma, la fecha de ingreso más antigua, la fecha de ingreso más reciente, el salario medio cobrado por los profesores y el salario total percibido por todos ellos.

```

SELECT count(*), min (fecha_ingreso), max (fecha_ingreso), avg (salario), sum (salario)
FROM profesores
    
```

Es preciso señalar algunas peculiaridades del comportamiento de las funciones agregadas en presencia de valores nulos. Así, en el caso de las funciones **SUM**, **AVG**, **MAX** y **MIN**, no se tienen en cuenta los valores nulos a la hora de hacer los cálculos.

El caso de la función **COUNT** se muestra con un ejemplo: Si se desean contar los registros totales que hay en una tabla con 100 filas, la sentencia **SELECT count (*) from tabla**, devolverá 100. Supóngase ahora que la tabla tiene un campo llamado **clave**. Si para dicho campo ninguna fila de la tabla toma el valor nulo, la sentencia **select count (clave) from tabla** devolvería también 100. Pero si hay valores nulos en el campo **clave** en alguna fila de la tabla, por ejemplo, en 7 de ellas, la sentencia **select count (clave) from tabla** devolvería 93, es decir, la función **count** aplicada sobre un campo concreto sólo cuenta las filas en las que dicho campo no es nulo.

La cláusula **WHERE** se usa para añadir condiciones a la búsqueda, así como para especificar los campos que se usan para hacer la unión (join) de tablas y/o vistas cuando hay más de una en la cláusula **FROM** (las uniones no son obligatorias, pero son necesarias, puesto que si no se especifican se realiza un producto cartesiano de las tablas, obteniéndose un conjunto de valores que no suele ser el deseado). Se pueden establecer condiciones complejas empleando los operadores **NOT**, **OR** y **AND**. Se muestra un ejemplo a continuación, en el que se puede además observar el empleo de alias tanto en las tablas presentes en la cláusula **FROM** como en algunos de los nombres de los campos recuperados:

```
SELECT p.nif, p.nombre_completo profesor, p.dirección, p.teléfono, d.nombre departamento
FROM profesores p, departamentos d
WHERE p.departamento_id = d.departamento_id
AND p.nombre_completo like '%PEREZ%'
```

La query anterior recupera información de dos tablas, *profesores* y *departamentos*, que se unen mediante el campo *departamento_id*, presente en ambas. De la tabla *profesores* se recuperan los campos *nif*, *nombre_completo* (al que se le pone el alias *profesor*), *dirección* y *teléfono*. De la tabla *departamentos* se recupera el campo *nombre*, al que se le pone el alias *departamento*. Además se especifica que sólo se recupere información de aquellos profesores cuyo nombre contiene la cadena de caracteres **PEREZ**.

La cláusula **GROUP BY** se emplea con las funciones agregadas cuando se quiere obtener ese tipo de información, no sobre el total de los registros que recupera la consulta, sino sobre grupos de éstos. El ejemplo siguiente muestra el nombre de cada departamento junto al salario medio y el número total de profesores de cada uno. Nótese que el campo (o campos) por el que se desea agrupar la información es el que se especifica en la cláusula **GROUP BY**.

```
SELECT d.nombre departamento, avg (p.salario), count(p.*)
FROM profesores p, departamentos d
WHERE p.departamento_id = d.departamento_id
GROUP BY d.nombre
```

La cláusula **HAVING** se emplea para poner condiciones a las funciones agregadas, del mismo modo que se usa la cláusula **WHERE** para poner condiciones a los campos de las tablas. Por ejemplo, si en la consulta anterior sólo se quisiera recuperar la información para aquellos departamentos que cuentan con un mínimo de tres profesores, la consulta quedaría del siguiente modo:

```
SELECT d.nombre departamento, avg (p.salario), count(p.*)
FROM profesores p, departamentos d
WHERE p.departamento_id = d.departamento_id
GROUP BY d.nombre
HAVING count (p.*) >= 3
```

La cláusula **ORDER BY** se usa para ordenar los resultados de la consulta de manera ascendente (**ASC**) o descendente (**DESC**). Se pueden poner los nombres de los campos por los que se desea ordenar o bien el orden en el que aparecen en la cláusula **SELECT**. Así, la consulta anterior, se

podría ordenar por número de profesores (de menor a mayor) y, para un mismo número de profesores, por media salarial (de mayor a menor) de la siguiente forma:

```
SELECT d.nombre departamento, avg (p.salario), count (p.*)
FROM profesores p, departamentos d
WHERE p.departamento_id = d.departamento_id
GROUP BY d.nombre HAVING count (p.*) >= 3
ORDER BY 2 ASC, 3 DESC
```

Cabe señalar que la cláusula **SELECT** admite dos modificadores, **ALL** (que es el que se usa por defecto y por eso casi nunca se escribe) y **DISTINCT**, que se usa cuando se quiere que la consulta no devuelva filas repetidas. Por ejemplo, la siguiente consulta devolvería los diferentes salarios de la tabla de *profesores*, de manera que si hay varios profesores con el mismo salario, esa cantidad no aparecería más que una vez (en caso de no ponerse el modificador **DISTINCT**, aparecería tantas veces como profesores tuvieran ese salario)

```
SELECT DISTINCT p.salario FROM profesores p
```

A continuación se muestra un ejemplo de la unión de dos consultas diferentes, tal y como en SQL se implementa la operación **UNION** del álgebra y el cálculo relacionales. Esta sentencia devolvería el NIF y el nombre contenidos en las tablas de profesores y catedráticos, de manera que si algún registro aparece en ambas tablas, sólo aparecería uno, el correspondiente a la primera (si se quiere que aparezcan todos, aunque estén repetidos en ambas ramas de la sentencia, en lugar de unirlos con la palabra clave **UNION**, se usaría **UNION ALL**)

```
SELECT nif, nombre FROM profesores
UNION
SELECT nif, nombre FROM catedraticos
```

La operación **INTERSECCION** del álgebra relacional se muestra en el ejemplo siguiente, que devolvería la lista de NIF y nombres que estuvieran presentes en las dos tablas implicadas en la consulta

```
SELECT nif, nombre FROM profesores
INTERSECT
SELECT nif, nombre FROM catedraticos
```

Por último se muestra un ejemplo de la operación **DIFERENCIA** del álgebra relacional, que devolvería los NIF y nombres presentes en la tabla *profesores* que NO estén en la tabla *catedraticos* (cabe señalar que el operador estándar es **EXCEPT**, pero algunos fabricantes implementan esta operación usando el operador **MINUS**)

```
SELECT nif, nombre FROM profesores
EXCEPT
SELECT nif, nombre FROM catedraticos
```

2.3.4 Lenguaje de manipulación de datos (DML)

2.3.4.1 Inserción de datos (INSERT)

El siguiente ejemplo muestra la sintaxis para la inserción de una nueva fila en base de datos. La sentencia sólo será dada por buena y ejecutada por el gestor de base de datos si la inserción de la fila en la tabla indicada no viola ninguna regla de integridad:

```
INSERT INTO profesores (nif, nombre_completo, dirección, teléfono, id_departamento)
VALUES ('222222222V', 'José Gómez', 'Calle Nueva, 10', 987654321, 17)
```

Pueden insertarse múltiples filas en una sola sentencia, bien indicando los valores a insertar directamente:

```
INSERT INTO profesores (nif, nombre_completo, dirección, teléfono, id_departamento)
VALUES (('222222222V', 'José Gómez', 'Calle Nueva, 10', 987654321, 17),
        ('333333333Z', 'Luisa González', 'Calle Antigua, 10', 98123456, 15))
```

O seleccionándolos de otra tabla donde estén cargados previamente:

```
INSERT INTO profesores (nif, nombre_completo, dirección, teléfono, id_departamento)
SELECT nif, nombre, dirección, tlfo, departamento
FROM empleados WHERE tipo_empleado = 'PROFESOR'
```

2.3.4.2 Actualización de datos (UPDATE)

El siguiente ejemplo muestra la sintaxis para la actualización de datos (en este caso, un aumento del 3% en el campo **salario** para los profesores del departamento 17). En el caso de esta sentencia, se actualizan todas las filas que cumplan la condición indicada en la cláusula **WHERE**. Si no se especifica dicha cláusula, la actualización alcanzaría a todas las filas sin excepción. Como en el caso anterior, la sentencia sólo se ejecuta si no viola ninguna regla de integridad.

```
UPDATE profesores
SET salario = salario * 1.03
WHERE id_departamento = 17
```

2.3.4.3 Actualización e inserción simultáneas referidas a otra tabla (MERGE)

Existe una sentencia en SQL que permite actualizar los registros de una tabla, de manera que aquellas filas que cumplan determinada condición se actualizan y, opcionalmente, aquellas filas que no la cumplan, se insertan completas desde otra tabla diferente.

Supóngase que se tienen en una tabla (**tabla_destino**) una serie de filas con información sobre la edad de un colectivo de personas, pero esa información está desactualizada. La información actualizada, junto con información de más personas no presentes en **tabla_destino** se encuentra en otra tabla diferente llamada **tabla_actualizacion**. Para, con una sola sentencia, actualizar la información existente en **tabla_destino** y al mismo tiempo insertar las filas que no están en dicha tabla, se haría lo siguiente:

```
MERGE INTO tabla_destino
USING tabla_actualizacion
ON tabla_destino.nif = tabla_actualizacion.nif
WHEN MATCHED THEN
    UPDATE SET tabla_destino.edad = tabla_actualizacion.edad
WHEN NOT MATCHED THEN
    INSERT (nif, edad) VALUES (tabla_actualizacion.nif, tabla_actualizacion.edad)
```

Como en los casos anteriores, la sentencia sólo se ejecuta si no viola ninguna regla de integridad.

2.3.4.4 Borrado de datos (DELETE y TRUNCATE)

El siguiente ejemplo muestra la sintaxis para el borrado de una fila o grupo de filas en base de datos. En el caso de esta sentencia, se eliminan todas las filas que cumplan la condición indicada en la cláusula **WHERE**. Si no se especifica dicha cláusula, la actualización alcanzaría a todas las filas sin excepción. Como en el caso anterior, la sentencia sólo se ejecuta si no viola ninguna regla de integridad. En este caso, se borran todos los profesores del departamento 15.

DELETE FROM profesores **WHERE** id_departamento = 15

TRUNCATE también se emplea para el borrado de datos, pero en realidad se trata de una sentencia DDL, por lo que se explicará más adelante, en el apartado correspondiente a dichas sentencias.

2.3.4.5 Confirmación y marcha atrás en sentencias DML

Cada sentencia o grupo de sentencias ejecutadas, es sólo visible para la conexión de base de datos desde la que se realiza. Para confirmar todas las sentencias DML previamente ejecutadas y que aún no habían sido ni confirmadas ni rechazadas, haciéndolas visibles para todos los usuarios de la base de datos se ejecuta la sentencia **COMMIT**.

Cuando se quiere dar marcha atrás (rechazar) una serie de operaciones DML, se debe ejecutar la operación **ROLLBACK**.

Estas dos operaciones facilitan el control de errores y el mantener la base de datos consistente. Cuando se ejecuta una tarea compleja que supone actualización, inserción y/o borrado mediante múltiples sentencias SQL, cabe la posibilidad de que en mitad de dichas operaciones se produzca algún error. En esos casos, se suele ejecutar un **ROLLBACK** para dejar la base de datos en el mismo estado que estaba cuando se iniciaron todas las operaciones y analizar el error. Si todo el conjunto de sentencias se ejecuta correctamente, se realiza un **COMMIT**, que confirma todos los cambios de una sola vez y los hace visibles de manera simultánea para el resto de usuarios.

2.3.5 Lenguaje de definición de datos (DDL)

Es el lenguaje que permite la creación, modificación y borrado no de datos (para eso está el DML), sino de los propios objetos que configuran la base de datos: tablas, vistas, índices, reglas de integridad, vistas materializadas, esquemas de datos, etc. Este es un lenguaje más complejo que el DML y que además tiene más particularidades en cada implementación, por lo que aquí se verán sólo las sentencias habituales y generales, aquellas que permiten la gestión de tablas, vistas, índices y reglas de integridad.

Es importante señalar que en el caso de las sentencias DDL, éstas se confirman de manera automática e inmediata, es decir, las sentencias **COMMIT** y **ROLLBACK** que se vieron en el apartado anterior no tienen efecto alguno sobre las sentencias DDL.

2.3.5.1 Creación, modificación y eliminación de tablas

Para crear una tabla se emplea la siguiente sentencia, en la que se indica el nombre de la tabla a crear y, entre paréntesis, la lista de campos que la componen y el tipo de datos de cada uno de ellos:

```
CREATE TABLE profesores (
    nif VARCHAR (10),
    nombre_completo VARCHAR (100),
    dirección VARCHAR (100),
    teléfono NUMBER (12),
    id_departamento NUMBER (3),
    salario NUMBER (8,2),
    CONSTRAINT profesores_pk PRIMARY KEY (nif),
    CONSTRAINT prof_dep_id_fk FOREIGN KEY (id_departamento)
```

REFERENCES (departamentos (id_departamento))

)

Al crear la tabla, además de los campos se están especificando dos reglas de integridad: la clave primaria de la tabla, que será el campo NIF, y una referencia de integridad, que especifica que los valores que en la tabla profesores tome el campo id_departamento deberán existir en la tabla de departamentos (en próximos apartados se verá como añadir reglas de integridad a tablas ya existentes, en caso de que no se haya hecho durante su creación)

Si ahora se quiere añadir un nuevo campo a la tabla para guardar la fecha de nacimiento:

ALTER TABLE profesores **ADD** fecha_nacimiento **DATE**

Y si se quiere borrar la tabla, incluyendo todos los datos que almacena, reglas de integridad, índices y cualquier otro objeto asociado a ella:

DROP TABLE profesores

Existe en DDL una sentencia que permite eliminar todas las filas de una tabla de una sola vez:

TRUNCATE TABLE profesores

Su efecto es exactamente el mismo que el de la sentencia DML **DELETE * FROM profesores**, con la diferencia de que al tratarse de una sentencia DDL, no tiene marcha atrás (**ROLLBACK**)

Es importante destacar que en SQL no es obligatorio que una tabla tenga clave primaria. Esto supone que pueda haber tuplas repetidas, cosa que la definición del modelo relacional no permite (como se comentaba en el apartado *Restricción de clave* de este mismo documento). Por tanto, en este sentido SQL es menos rígido que el modelo relacional.

2.3.5.2 Creación, modificación y eliminación de vistas

La sintaxis es muy similar al caso de las tablas, salvo que las vistas, en su creación, se especifican mediante una sentencia **SELECT**. El siguiente ejemplo crea una vista con el nif y nombre de los profesores y el nombre del departamento al que pertenecen, siguiendo la línea de los ejemplos anteriores:

```
CREATE VIEW profesores_v AS  
SELECT p.nif, p.nombre_completo profesor, d.nombre departamento  
FROM profesores p, departamentos d  
WHERE p.departamento_id = d.departamento_id
```

Las vistas no se suelen modificar, sino que se suelen crear de nuevo, al no tratarse de objetos físicos en base de datos, sino de simples referencias. Para añadir la dirección a la vista anterior se haría:

```
CREATE OR REPLACE VIEW profesores_v AS  
SELECT p.nif, p.nombre_completo profesor, p.direccion, d.nombre departamento  
FROM profesores p, departamentos d  
WHERE p.departamento_id = d.departamento_id
```

Y para eliminar una vista

DROP VIEW profesores_v

2.3.5.3 Creación y eliminación de índices

Las dos siguientes sentencias muestran la creación de dos índices, el primero de ellos único. Se especifica el nombre de la tabla y del campo concreto sobre el que se crea el índice:

```
CREATE UNIQUE INDEX i_dep_id ON departamentos (id_departamento)
CREATE INDEX i_pro_dep_id ON profesores (id_departamento)
```

La diferencia entre las dos sentencias anteriores es que la primera, al crear un índice único, no va a permitir que el campo indexado (*id_departamento*) tenga valores repetidos, mientras que la segunda, al no indicar que el índice es único, sí lo permitiría.

Para borrar un índice:

```
DROP INDEX i_dep_id
```

2.3.5.4 Creación y eliminación de reglas de integridad

Añadir una clave primaria a una tabla cuando ésta ya existe:

```
ALTER TABLE profesores
ADD CONSTRAINT profesores_pk PRIMARY KEY (nif)
```

Añadir una clave ajena (foreign key) a una tabla cuando ésta ya existe:

```
ALTER TABLE profesores
ADD CONSTRAINT profesores_dep_id_fk
FOREIGN KEY (departamento_id)
REFERENCES departamentos (departamento_id)
```

Para eliminar reglas de integridad, sean de un tipo o de otro:

```
ALTER TABLE profesores
DROP CONSTRAINT profesores_dep_id_fk,
DROP CONSTRAINT profesores_pk
```

2.3.6 Lenguaje de control de datos (DCL)

Se emplea para asignar diferentes niveles de permisos de acceso y/o manipulación sobre los objetos de la base de datos a los distintos usuarios de la misma. Las dos sentencias más importantes son **GRANT** (para la concesión de permisos) y **REVOKE** (para su revocación)

Las operaciones contempladas por el estándar sobre las que se pueden conceder o revocar permisos son inserción (**INSERT**), actualización (**UPDATE**), borrado (**DELETE**), consulta (**SELECT**), ejecución de código almacenado en base de datos (**EXECUTE**), conexión a un esquema o usuario de la base de datos (**CONNECT**) y conexión a la propia base de datos sin ningún privilegio adicional (**USAGE**).

Las distintas implementaciones de SQL permiten añadir mayor granularidad a estas operaciones y suelen incluir también privilegios para operaciones DDL (**CREATE**, **DROP**, etc.) y para las propias operaciones DCL.

La sintaxis, tanto para conceder como para revocar privilegios para estas operaciones es muy similar.

```
GRANT <operación> ON <nombre_de_objeto_sobre_el_que_se_da_permiso>
TO <nombre_de_usuario>
REVOKE <operación> ON <nombre_de_objeto_sobre_el_que_se_revoca_permiso>
```

FROM <nombre_de_usuario>

2.4 Extensiones procedurales

SQL es un lenguaje creado con un propósito muy específico: consultar y gestionar bases de datos relacionales, tanto su estructura como la información que almacenan. Al no tratarse, por tanto, de un lenguaje de propósito general, carece de las estructuras necesarias para realizar verdaderos programas de aplicación (no hay bucles, sentencias condicionales, ni tampoco la posibilidad real de escribir programas, más allá de las propias sentencias sueltas del lenguaje)

Por ello, diversos fabricantes de sistemas gestores de bases de datos relacionales e intérpretes de SQL han creado y soportan diferentes lenguajes de programación, denominados extensiones procedurales, que se integran con SQL y que permiten construir programas completos en dichos lenguajes, específicamente diseñados además para que el SQL del fabricante se integre de manera natural en los mismos.

Algunas de estas extensiones se enumeran a continuación, junto a una breve reseña de las mismas:

- SQL/PSM: Publicada por ISO, es parte del estándar ISO 9075, aunque no es de implementación obligatoria. Las extensiones procedurales de MySQL y SQL PL (de IBM) son las implementaciones más cercanas a este estándar. También cabe citar otras implementaciones como las de Mimer SQL, Monet DB, o PL/PSM (publicada por Postgre)
- SPL, publicado por IBM para la base de datos Informix.
- T-SQL, publicado por Microsoft para Sybase.
- PL/SQL de Oracle.
- PL/pgSQL, también de Postgre, basado en Oracle PL/SQL.
- SQL HANA de SAP.

2.5 Conclusiones

De cara al estudio de este apartado deben conocerse los tipos de datos contemplados en SQL, así como los operadores y la sentencia CASE. Debe conocerse y entenderse el funcionamiento de la sentencia SELECT, así como de las demás sentencias DML y DDL (en este caso sí sería especialmente útil, si no se tiene experiencia y/ conocimientos previos de SQL, disponer de una base de datos que soporte dicho lenguaje y contra la que se puedan probar las distintas sentencias y ver su funcionamiento real, contrastándolo con lo que uno espera encontrarse en cada caso).

Las sentencias DCL, así como las extensiones procedurales deben conocerse, pero no es necesario profundizar demasiado en ellas.

3 Normas y estándares para la interoperabilidad entre gestores de bases de datos relacionales

3.1 Introducción

La amplia variedad de fabricantes de sistemas gestores de base de datos, unida a la enorme disparidad de lenguajes de programación disponibles para el desarrollo de aplicaciones que precisan acceder a bases de datos, ha llevado a la necesidad de idear mecanismos que permitan hacer a los desarrolladores lo más transparente posible la tarea de acceder a las distintas bases de datos de los distintos fabricantes y a integrar de la manera más transparente posible esos accesos dentro de sus programas de aplicación, independientemente del lenguaje en el que estén desarrollados.

En los próximos apartados se enumeran y describen brevemente algunos de estos mecanismos.

3.2 Primeros esfuerzos y aparición de ODBC

A finales de la década de los 80, se hicieron los primeros intentos de crear interfaces de programación genéricas para acceso a diferentes gestores de base de datos.

Entre estos primeros trabajos cabe destacar, dentro del ámbito de los mainframes, DRDA (Distributed Relational Database Architecture) de IBM o DAL (Data Access Language) de Apple.

En la misma línea, pero pensando en microcomputadores, se pueden citar Blueprint, desarrollado por Lotus para permitir acceso desde su software de hoja de cálculo Lotus 1-2-3 a diversos orígenes de datos SQL, o SQLC (SQL Connectivity), el primer estándar en el que hubo una amplia colaboración de gran parte de la industria.

En 1988, Microsoft, Lotus, DEC y Sybase dan un importante impulso a SQLC que, si bien después de ver reducido en más de un 50% su contenido inicial, pasa a ser adoptado por ISO 9075 como parte del estándar SQL, denominándose SQL/CLI (Call Level Interface)

Microsoft decide continuar por su cuenta el trabajo con SQLC sin los recortes que acabaron siendo el estándar SQL/CLI y en 1992 presenta como resultado de su trabajo ODBC 1.0, estructurado en tres niveles. El nivel 0, o Core, es exactamente igual al estándar SQL/CLI. El nivel 1 está formado por comandos adicionales fácilmente implementables en drivers y el nivel 2 soporta las características más avanzadas.

En 1993 Openlink Software es la primera compañía independiente que publica drivers ODBC para sistemas de base de datos de terceras partes (PROGRESS DBMS, Sybase, Oracle...)

A pesar de que Microsoft abandonó el desarrollo de ODBC para trabajar en otros paradigmas, tales como OLE DB, ADO y ADO.NET, el mundo del software libre, especialmente desde la aparición de PostgreSQL y MySQL, lo ha adoptado como estándar de acceso a bases de datos.

Por su parte, Sun Microsystems (posteriormente adquirida por Oracle) usó ODBC como base para el desarrollo de su propio estándar abierto, JDBC.

En la actualidad, con la proliferación de los desarrollos basados en clientes ligeros y HTML ha desaparecido la necesidad de acceso directo a la base de datos por parte de dichos clientes, por lo que los trabajos sobre ODBC han dejado de ser un interés primordial para la industria.

3.3 OLE DB (Object Linking and Embedding Database), ADO (Active X Data Objects) y ADO.NET

OLE DB es una tecnología desarrollada por Microsoft como sucesora de ODBC para el acceso a bases de datos de diferentes tipos de una manera uniforme, incluyendo fuentes de datos no relacionales, tales como bases de datos de objetos, hojas de cálculo, etc.

Otra de sus novedades es su arquitectura en capas, que permite un nivel de abstracción suficiente para independizar las aplicaciones del acceso a los datos que las mismas necesitan.

SQL Server 2012 es la última versión de este producto que implementa OLE DB, cuyo soporte por parte de Microsoft se prolongará durante un periodo de siete años más.

ADO es también un desarrollo de Microsoft creado con la misión de proporcionar un middleware que se sitúe entre las aplicaciones y la tecnología OLE DB. En este caso, el programador de aplicaciones sólo debe gestionar la conexión de base de datos, no necesita programar en SQL para acceder o manipular la información.

ADO.NET ha sido la evolución de ADO al desarrollar Microsoft la tecnología .NET, si bien son tantos los cambios en el propio producto (aunque su filosofía sigue siendo la de proporcionar una abstracción total al desarrollador de aplicaciones sobre las particularidades de la base de datos a la que accede) que puede considerarse un desarrollo per sé. Sus principales aportaciones son el soporte completo a XML, que se basa en un modelo desconectado (al contrario que ADO), es decir, los datos se recuperan, el cliente se desconecta de la base de datos, trabaja sobre los datos y sólo vuelve a conectarse cuando va a actualizar la información tras finalizar su procesamiento.

3.4 JDBC (Java Database Connectivity)

Es una API para acceso a bases de datos desde el lenguaje de programación Java. JDBC está orientado a bases de datos relacionales, pero puede acceder a otras fuentes de datos mediante un puente JDBC-ODBC.

Existen cuatro tipos de drivers JDBC (adaptadores que se instalan en el lado del cliente)

3.4.1 Tipo 1: Puente JDBC-ODBC

En este caso es el driver ODBC el que se conecta a la base de datos, mientras que el driver tipo 1 traduce las llamadas JDBC que realizan las aplicaciones en llamadas ODBC. El esquema de esta implementación es el siguiente:

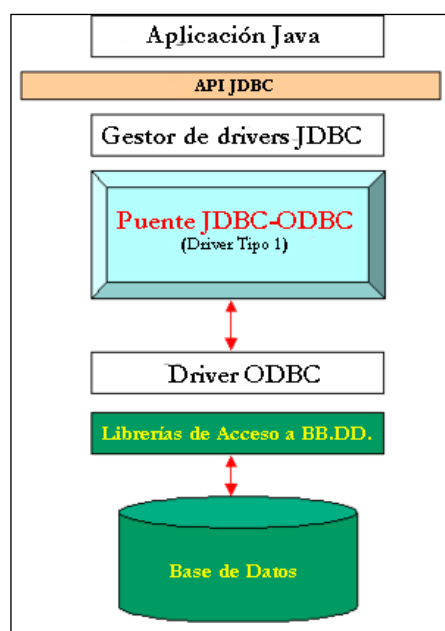


Figura 1. Driver JDBC Tipo 1

La principal ventaja de este tipo de driver es que permite acceder a prácticamente cualquier base de datos para la que exista un driver ODBC.

Sus principales desventajas son su bajo rendimiento, al haber múltiples capas y que los drivers ODBC tienen que estar instalados en la máquina cliente, lo cual añade una tercera desventaja, no puede usarse para implementar Applets.

3.4.2 Tipo 2: API Nativa

En este caso se emplean librerías de base de datos instaladas en el cliente. El driver tipo 2 convierte las llamadas JDBC en llamadas a esas librerías. El esquema de esta implementación es el siguiente:

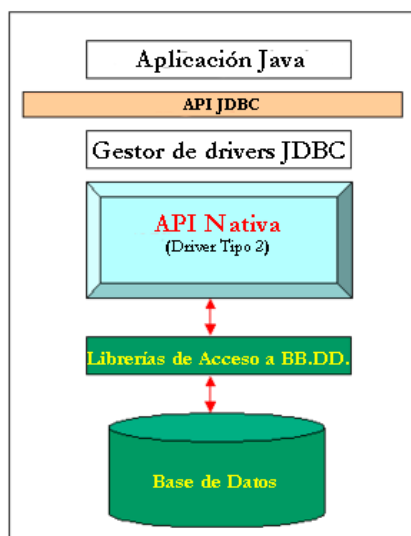


Figura 2. Driver JDBC Tipo 2

La principal ventaja de este tipo de driver es su rapidez, al desaparecer el puente JDBC-ODBC.

Sus principales desventajas son que las librerías de acceso a base de datos deben estar instaladas en el cliente (no todas las bases de datos disponen de tales librerías), que el driver es dependiente de la plataforma y que, por el mismo motivo que con el driver tipo 1, no es válido para Applets.

3.4.3 Tipo 3: Protocolo de red

En este caso el driver envía las llamadas a un middleware instalado en el servidor que se encarga de traducir las llamadas JDBC para que sean entendidas por las distintas bases de datos a las que se accede. Además de la denominación que se ha empleado aquí para este driver, **protocolo de red**, en los textos que tratan este tema pueden encontrarse otras, tales como **Frontend que accede al middleware** o **Driver Java Puro para Middleware de Base de Datos**. El esquema de esta implementación es el siguiente:

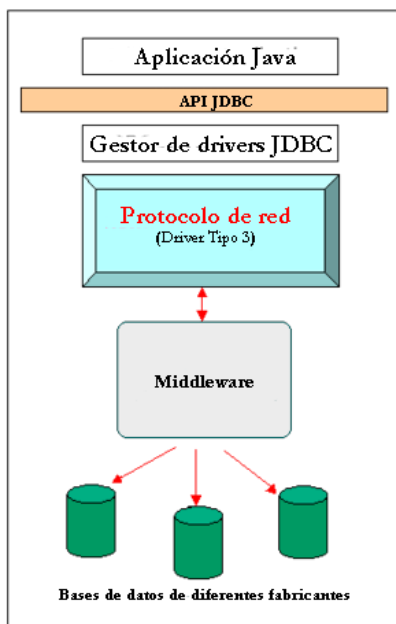


Figura 3. Driver JDBC Tipo 3

Las principales ventajas de este tipo de driver son que no necesita una instalación en cliente específica para cada base de datos, sigue un modelo de arquitectura de tres capas y el driver está escrito 100% en Java y se comunica con el middleware, no siendo por tanto dependiente de la base de datos.

Sus principales desventajas son que necesita de programación específica para cada base de datos en el middleware y que ésta capa podría introducir algo de retardo sin unos buenos servicios de middleware.

3.4.4 Tipo 4: Java puro

También llamado **driver nativo en Java**, en este caso el driver convierte directamente las llamadas JDBC al protocolo de la base de datos. Proporciona mejores prestaciones que los drivers tipo 1 y 2 al no haber ninguna capa adicional y, al contrario que los de tipo 3, no necesita de ningún software adicional para funcionar. El esquema de esta implementación es el siguiente:

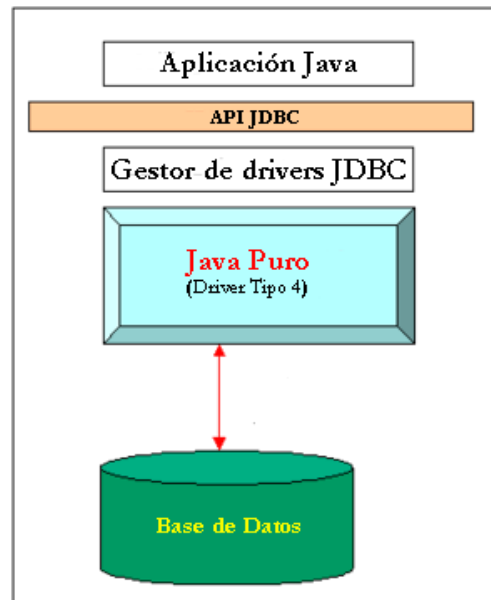


Figura 4. Driver JDBC Tipo 4

Las principales ventajas de este tipo de driver son su implementación 100% Java, lo que lo hace totalmente independiente de la plataforma, el no uso de ninguna capa intermedia, lo que aumenta el rendimiento y que al ser la máquina virtual de Java la que gestiona toda la conexión, es más sencilla la depuración de los programas.

Su principal desventaja es la necesidad de disponer de un driver específico para cada fabricante de base de datos.

3.5 Conclusiones

En este apartado el punto más importante es el último, donde se explican los distintos tipos de drivers JDBC. Es importante conocer este apartado a fondo y saber para qué pueden o no utilizarse los distintos tipos de drivers y sus ventajas/inconvenientes. También debe conocerse la existencia de otras tecnologías de acceso a bases de datos (JDBC es exclusivo para Java) y su evolución en el tiempo, si bien no es tan importante entrar tanto en detalle.

4 Principales gestores de bases de datos relacionales

Según DB-Engines, organización que elabora un ranking de las bases de datos más populares del mundo, en Mayo de 2015 los cinco sistemas de gestión de bases de datos relacionales más populares eran Oracle, MySQL, SQL Server, PostgreSQL y DB2.

Se muestran a continuación algunas tablas comparativas de los cinco con datos de abril de 2015.

4.1 Fabricante, última versión y tipo de licencia

Base de Datos	Fabricante	Versión Estable	Fecha Versión	Tipo de Licencia
Oracle	Oracle	12c Release 1	Jun-2013	Propietaria
MySQL	Oracle	5.6.22	Dic-2014	GPLv2 o Propietaria
SQL Server	Microsoft	2014	Mar-2014	Propietaria
PostgreSQL	PostgreSQL Global Development Group	9.4.0	Dic-2014	PostgreSQL license (open source)
DB2	IBM	10.5	Abr-2013	Propietaria

Tabla 1. Fabricante, última versión y tipo de licencia

4.2 Sistemas operativos principales en los que está disponible

Base de Datos	Windows	OS X	Linux	BSD	UNIX	zOS
Oracle	Sí	Sí	Sí	No	Sí	Sí
MySQL	Sí	Sí	Sí	Sí	Sí	Sí
SQL Server	Sí	No	No	No	No	No
PostgreSQL	Sí	Sí	Sí	Sí	Sí	Bajo Linux
DB2	Sí	Sí	Sí	No	Sí	Sí

Tabla 2. Sistemas operativos principales en los que está disponible

4.3 Límites de tamaño de la propia base de datos

Base de Datos	Tamaño de la base de datos completa	Tamaño de cada tabla individual	Tamaño de cada fila dentro de una tabla	Número de columnas por tabla	Caracteres en el nombre de columnas
Oracle	Ilimitado	4 Gigabytes * tamaño de bloque	8 KiloBytes	1.000	30
MySQL	Ilimitado	256 TeraBytes	64 KiloBytes	4.096	64
SQL Server	524.272 TeraBytes	524.272 TeraBytes	8.060 Bytes	30.000	128
PostgreSQL	Ilimitado	32 TeraBytes	1.6 TeraBytes	256-1600	63
DB2	Ilimitado	2 ZBytes ($2 \cdot 10^{21}$)	32.677 Bytes	1.012	128

Tabla 3. Límites de tamaño de la propia base de datos

4.4 Límites de tamaño de los tipos de datos principales

Base de Datos	Blob/Clob	CHAR	NUMBER	Fecha mínima	Fecha máxima
Oracle	128 TeraBytes	32 KiloBytes	126 bits	-4712	9999
MySQL	4 GigaBytes	64 Kilobytes	64 bits	1000	9999
SQL Server	2 GigaBytes	2 GigaBytes	126 bits	0001	9999
PostgreSQL	4 TeraBytes	1 GigaByte	Ilimitado	-4713	5.874.897
DB2	2 GigaBytes	32 Kibibytes	64 bits	0001	9999

Tabla 4. Límites de tamaño de los tipos de datos principales

4.5 Conclusiones

En este caso la información se aporta más como una referencia que como algo que haya que memorizar (sería prácticamente imposible hacerlo, entre otras cosas porque cada nueva versión del producto de cada fabricante puede cambiar la información que se presenta en estas tablas). Si es interesante en cualquier caso saber la disponibilidad de cada producto en los diferentes sistemas operativos y el tipo de licencia bajo el que hay que utilizarlos.

GLOSARIO

Se muestra a continuación una lista con las siglas aparecidas a lo largo del texto y su significado.

ADO: ActiveX Data Objects (Objetos de Datos ActiveX)

ANSI: American National Standards Institute (Instituto Nacional Americano de Estándares)

DAL: Data Access Language (Lenguaje de Acceso a Datos)

DB2: No se corresponde con unas siglas, es el gestor de base de datos SQL de IBM.

DCL: Data Control Language (Lenguaje de Control de Datos)

DDL: Data Definition Language (Lenguaje de Definición de Datos)

DEC: Digital Equipment Corporation (Nombre Comercial)

DML: Data Manipulation Language (Lenguaje de Manipulación de Datos)

DRDA: Distributed Relational Database Architecture (Arquitectura de Base de Datos Distribuida)

HTML: Hypertext Markup Language (Lenguaje de Marcado de Hipertexto)

IBM: Industrial Business Machine Corporation (Nombre comercial)

IEC: International Electrotechnical Commission (Comisión Electrotécnica Internacional)

ISO: International Organization for Standardization (Organización Internacional para la Estandarización)

JDBC: Java Database Connectivity (Conectividad con Base de Datos para Java)

JTC: Joint Technical Committee (Comité Técnico Conjunto)

ODBC: Open Database Connectivity (Conectividad Abierta para Base de Datos)

OLE-DB: Object Linking and Embedding – Database (Enlace e Inclusión de Objetos – Base de Datos)

SQL: Structured Query Language (Lenguaje Estructurado de Consulta)

SQL/CLI: SQL Call Level Interface (SQL Interface a Nivel de Llamada)

SQLC: SQL Connectivity (Conectividad SQL)

VAX: Virtual Address Extension (Nombre de una serie de servidores y grandes equipos de DEC)

XML: Extensible Markup Language (Lenguaje Extensible de Marcado)

PREGUNTAS DE TEST

- 1) ¿Cuál de las siguientes no es una característica de las tablas de una base de datos relacional?
 - a) El orden de las filas no es relevante.
 - b) Los dominios de las columnas pueden coincidir.
 - c) La intersección de una fila y una columna puede contener más de un valor.
 - d) El orden de las columnas no es relevante.
- 2) En el contexto del modelo relacional, si Operador1 = Verdadero y Operador2 es nulo. ¿Cuál de las siguientes operaciones da como resultado *Verdadero*?
 - a) NOT (Operador1 AND Operador2)
 - b) NOT Operador2.
 - c) (Operador1 OR Operador2) OR NOT Operador2.
 - d) NOT Operador1 OR NOT Operador2.
- 3) En el modelo relacional, respecto de las claves primarias y las claves candidatas de una tabla:
 - a) Toda clave candidata es también clave primaria.
 - b) Toda clave primaria es también clave candidata.
 - c) a) y b) son correctas.
 - d) a) y b) son falsas.
- 4) En el contexto del modelo relacional, si la tabla R2 tiene un campo A2 sobre el que hay definida una clave foránea contra la clave primaria de la tabla R1 (campo A1). Señale la respuesta correcta.
 - a) El campo A2 de R2 podría ser nulo y el campo A1 de R1 no puede serlo nunca.
 - b) El campo A1 de R1 podría ser nulo para algunas filas.
 - c) Ni el campo A1 de R1 ni el campo A2 de R2 pueden ser nulos en ninguna fila.
 - d) Las tres afirmaciones anteriores son falsas.
- 5) Según el modelo relacional, en una relación con tres atributos, el número máximo de superclaves posibles es:
 - a) 1
 - b) 2
 - c) 3
 - d) Todas las respuestas anteriores son falsas.
- 6) Según el modelo relacional, en una relación con tres atributos, el número de claves primarias es:
 - a) Depende de la relación.
 - b) c) y d) son incorrectas.
 - c) 1
 - d) 3
- 7) En el contexto del modelo relacional, señale cuál de los siguientes términos no es sinónimo de los demás:
 - a) Diccionario de datos.
 - b) Metadatos.
 - c) Catálogo.
 - d) Inventario de objetos.
- 8) Según las reglas de Codd:
 - a) Los valores nulos representan la falta de información o información desconocida.
 - b) El sistema gestor debe ser capaz de actualizar todas las vistas sean o no teóricamente actualizables.
 - c) El sistema gestor proporciona operadores para recuperar múltiples filas, pero el resto de operaciones sólo es preciso soportarlas fila a fila.
 - d) Los lenguajes de bajo nivel pueden emplearse para evitar restricciones de integridad.

- 9) Según el modelo relacional, en una relación con n atributos, el número máximo de atributos principales es:
- a) $n!$
 - b) n
 - c) $n/2$
 - d) 1
- 10) Respecto de la normalización de bases de datos relacionales, es falso que:
- a) Su objetivo principal es reducir la redundancia de datos.
 - b) Es una técnica empleada durante el diseño de la base de datos.
 - c) Es un proceso que se realiza en fases sucesivas.
 - d) Métrica 3 considera suficiente normalizar hasta la tercera forma normal.
- 11) Respecto de la Forma Normal de Boyce-Codd (FNBC)
- a) Es una variante algo más restrictiva de la Tercera Forma Normal.
 - b) Es una variante algo más restrictiva de la Cuarta Forma Normal.
 - c) No tiene relación con ninguna de las otras formas normales.
 - d) Todas las respuestas anteriores son falsas.
- 12) Respecto del álgebra relacional:
- a) Sus operaciones producen siempre como resultado una relación.
 - b) Sus operadores son siempre binarios (requieren dos operandos)
 - c) a) y b) son correctas.
 - d) a) y b) son falsas.
- 13) En el contexto del álgebra relacional:
- a) La operación Unión requiere que sus dos operandos sean compatibles.
 - b) La operación Diferencia de Conjuntos requiere que sus dos operandos sean compatibles.
 - c) a) y b) son correctas
 - d) a) y b) son falsas
- 14) La operación Producto Cartesiano en álgebra relacional devuelve una relación cuyo número de tuplas es:
- a) Siempre el producto del número de tuplas de cada uno de sus operandos.
 - b) Siempre la suma del número de tuplas de cada uno de sus operandos.
 - c) Nunca puede ser una relación vacía.
 - d) Depende de los valores de los atributos de ambas relaciones.
- 15) La operación Intersección en álgebra relacional devuelve una relación cuyo número de tuplas es:
- a) Siempre el producto del número de tuplas de cada uno de sus operandos.
 - b) Siempre la suma del número de tuplas de cada uno de sus operandos.
 - c) Nunca puede ser una relación vacía.
 - d) Depende de los valores de los atributos de ambas relaciones.
- 16) En SQL, la sentencia SELECT:
- a) Permite obtener filas individuales de las tablas, pero nunca realizar operaciones con sus valores.
 - b) Recupera siempre tablas completas, el filtrado de datos se realiza por los programas de aplicación que la invocan.
 - c) Dispone de funciones agregadas, entre otras, de una para contar filas de tablas que cumplan determinadas condiciones.
 - d) No dispone de operadores para buscar patrones de texto.
- 17) En SQL, y dada una tabla relacional t no vacía con tres campos a , b y c , siendo a la clave primaria:
- a) *Delete * from t where a is not null* podría no borrar ninguna fila, borrar algunas filas o borrar todas las filas de t .

- b) *Delete * from t where a is null* podría no borrar ninguna fila, borrar algunas filas o borrar todas las filas de *t*.
 - c) *Delete * from t* borra todas las filas de *t*, siendo esta operación inmediatamente visible en todo caso para todos los usuarios de la base de datos.
 - d) *Truncate table t* borra todas las filas de *t*, siendo esta operación inmediatamente visible en todo caso para todos los usuarios de la base de datos.
- 18) En SQL, y dada una tabla relacional *t* no vacía con tres campos *a*, *b* y *c*, siendo *a* la clave primaria; señale la respuesta falsa.
- a) *Select Count(*) from t* podría ser diferente de *Select Count(a) from t*.
 - b) *Select Count(*) from t* podría ser diferente de *Select Count(b) from t*.
 - c) *Select Count(b) from t* podría ser diferente de *Select Count(distinct b) from t*.
 - d) *Select Count(*) from t where a is null* nunca puede ser cero salvo que *t* estuviera vacía.
- 19) En SQL, para añadir una regla de integridad a una tabla ya existente:
- a) Se emplea la sentencia ALTER TABLE...ADD CONSTRAINT...
 - b) Las claves primarias sólo pueden especificarse en el momento de crear la tabla. Para las claves foráneas se emplearía la sintaxis mostrada en a)
 - c) Ningún tipo de regla de integridad puede crearse sobre tablas ya existentes.
 - d) Todas las respuestas son falsas.
- 20) Respecto de los distintos tipos de driver JDBC:
- a) El Tipo 4 es el más eficiente por tener menos capas.
 - b) El Tipo 3 no es posible usarlo con Applets.
 - c) El Tipo 2 es el idóneo para Applets.
 - d) El Tipo 1 es 100% Java.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	C
2	C
3	B
4	A
5	D
6	C
7	D
8	A
9	B
10	B
11	A
12	A
13	C
14	A
15	D
16	C
17	D
18	A
19	A
20	A

BIBLIOGRAFÍA

1. A. Silberschatz, H. F. Korth y S. Sudarshan. Fundamentos de Bases de Datos (4ª edición). McGraw Hill, 2002.
2. Francisco Charte Ojeda. SQL. Anaya Multimedia, 2014.
3. R. Elmasri y S. Navathe. Fundamentos de los Sistemas de Bases de Datos (3ª edición). Addison-Wesley, 2002.